

Introduction to string indexing

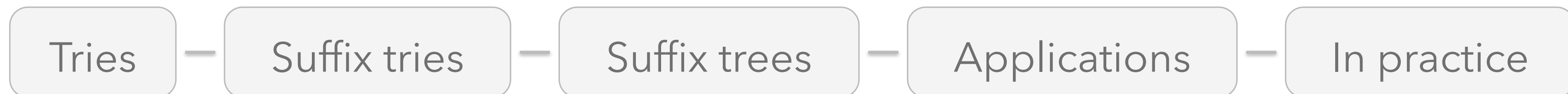
CMSC 423

Preprocess the text once, then answer every query in time proportional to the query



Where we are going

Every structure today answers the same question – “where does P occur in T ?” – by preprocessing T once, up front.



All five are today. The first three are the structures themselves; the last two are what you can do with them, and what they cost on real data.

- **The trade we are making:** spend time and space once, on T alone, so that every later query is cheap.
- **The tension we will hit:** the most natural index is quadratic in $|T|$ – unusable for a genome. Fixing that is the story of the second half.

Notation

symbol	meaning	in today's running example
<i>T</i>	the text – indexed once, in advance	<i>T</i> = abaaba\$
<i>m</i>	the length of the text, $ T $	<i>m</i> = 7
<i>P</i> or <i>S</i>	the query – a pattern we look for in T	<i>S</i> = aba
<i>n</i>	the length of the query, $ P $	<i>n</i> = 3
Σ	the alphabet T and P are drawn from	Σ = {a, b}
<i>k</i>	the number of occurrences we have to report	<i>k</i> = 2
\$	a terminator: a character appearing nowhere else in T	<i>smaller than every other character</i>

Indices into T are 0-based; $T[i..j]$ includes both ends. Every suffix trie and suffix tree in this lecture is built over T with $\$$ already appended.

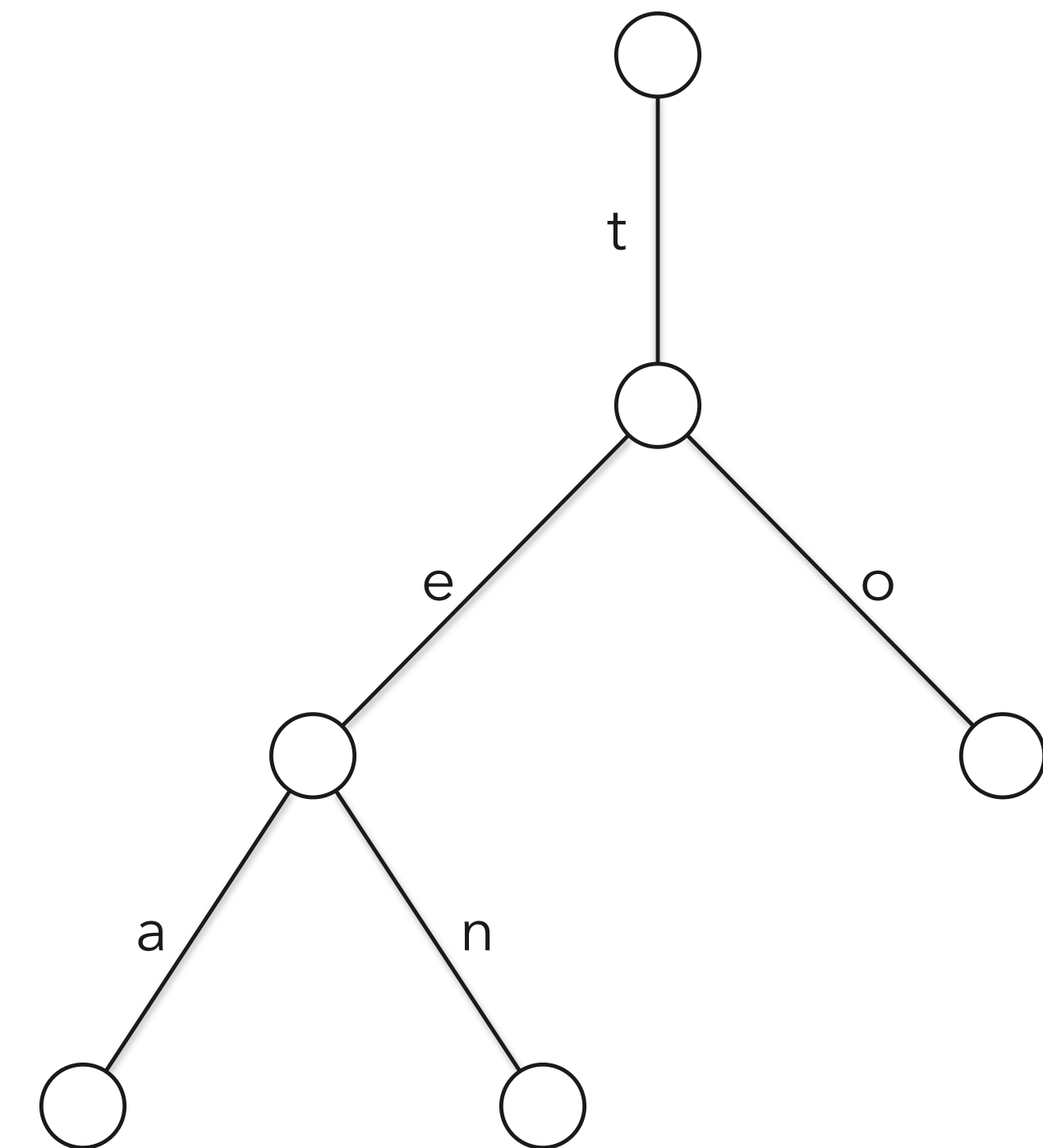
Tries

One node per common prefix

A trie

- A **trie** (rhymes with “try”) is a **rooted** tree representing a collection of strings, with one node per distinct common prefix.
- It is the smallest tree such that:
 - each edge is labelled with a single character $c \in \Sigma$
 - no node has two outgoing edges labelled with the same c
 - each key is spelled out along a path from the root
- A natural representation of a *set* or a *map* whose keys are strings.

Also known as a Σ -tree: every edge carries one symbol of the alphabet Σ .



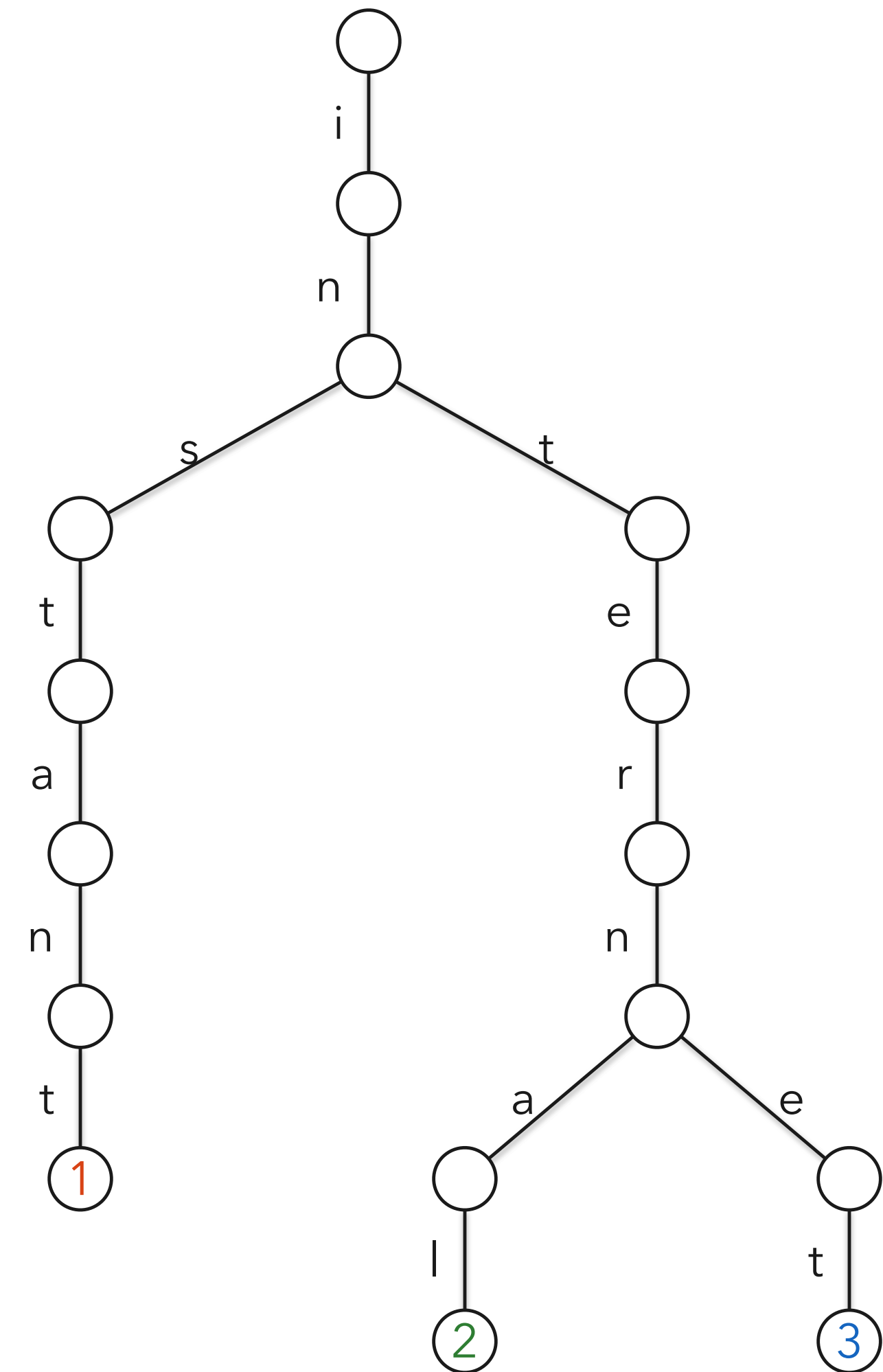
keys: to, tea, ten

A trie: example

Represent this map with a trie:

key	value
instant	1
internal	2
internet	3

- Internal nodes carry no value – they exist only because some keys share a prefix.
- Only the nodes drawn in color are keys. *“inter” is a node, but it is not a key.*



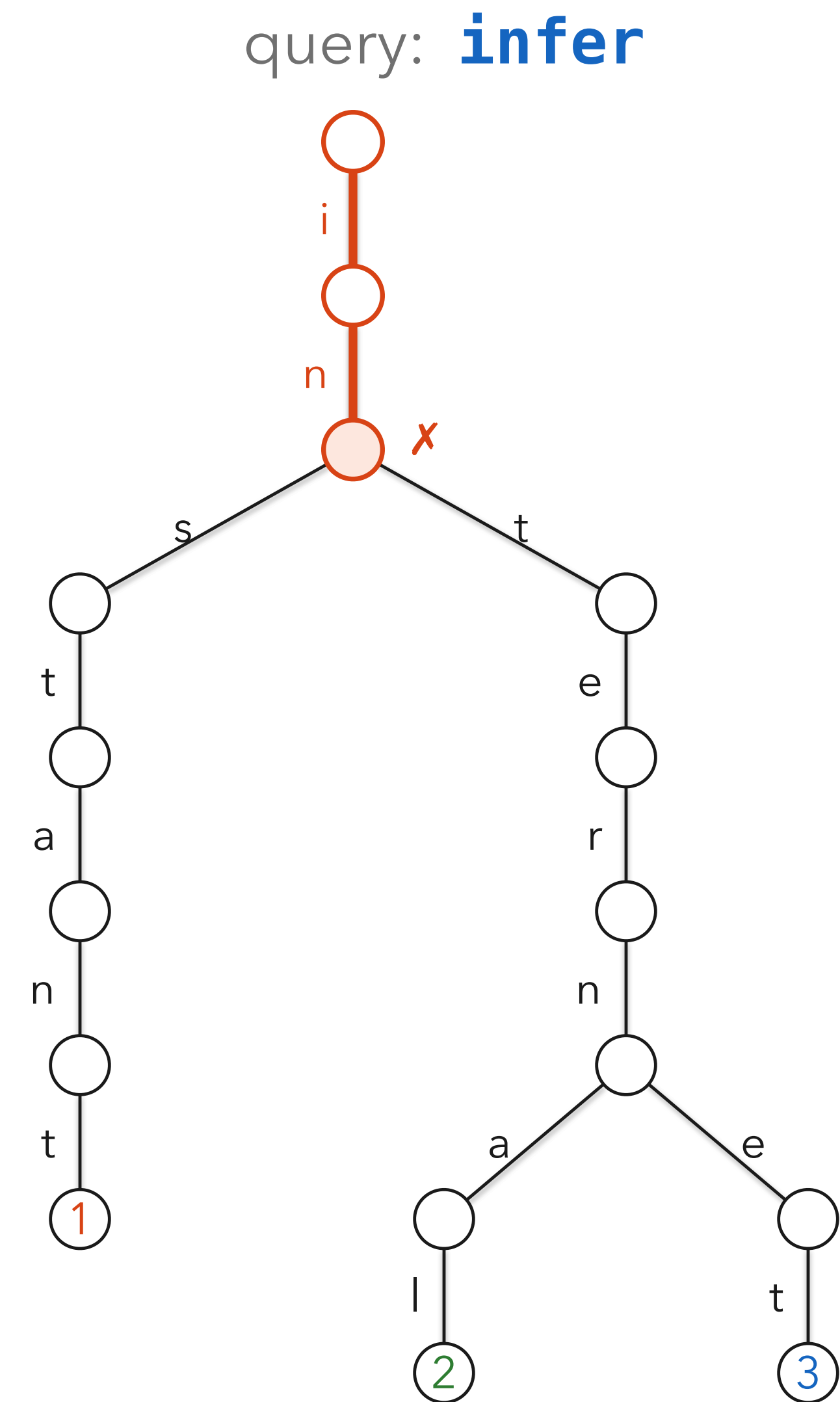
Tries: matching "infer"

How do we check whether a key is in the trie?

Start at the root and match successive characters of the query to edge labels.

matched: **infer**

no "f" out of "in" –
fall off the trie



Tries: matching "interesting"

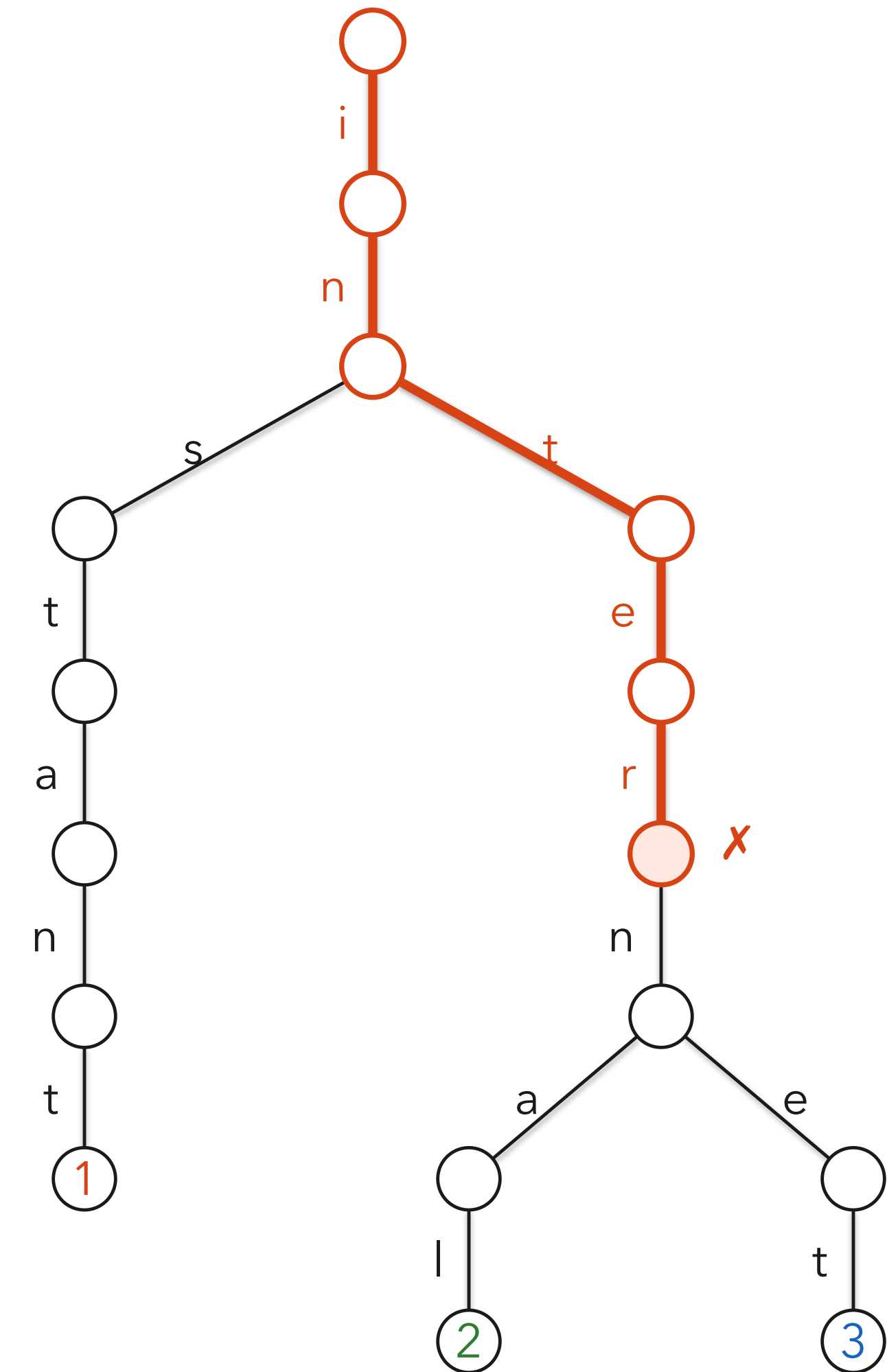
query: **interesting**

Matching "interesting"

Five characters match – i, n, t, e, r – and then the trie has no "e" out of "inter".

matched: **interesting**

no "e" out of "inter"



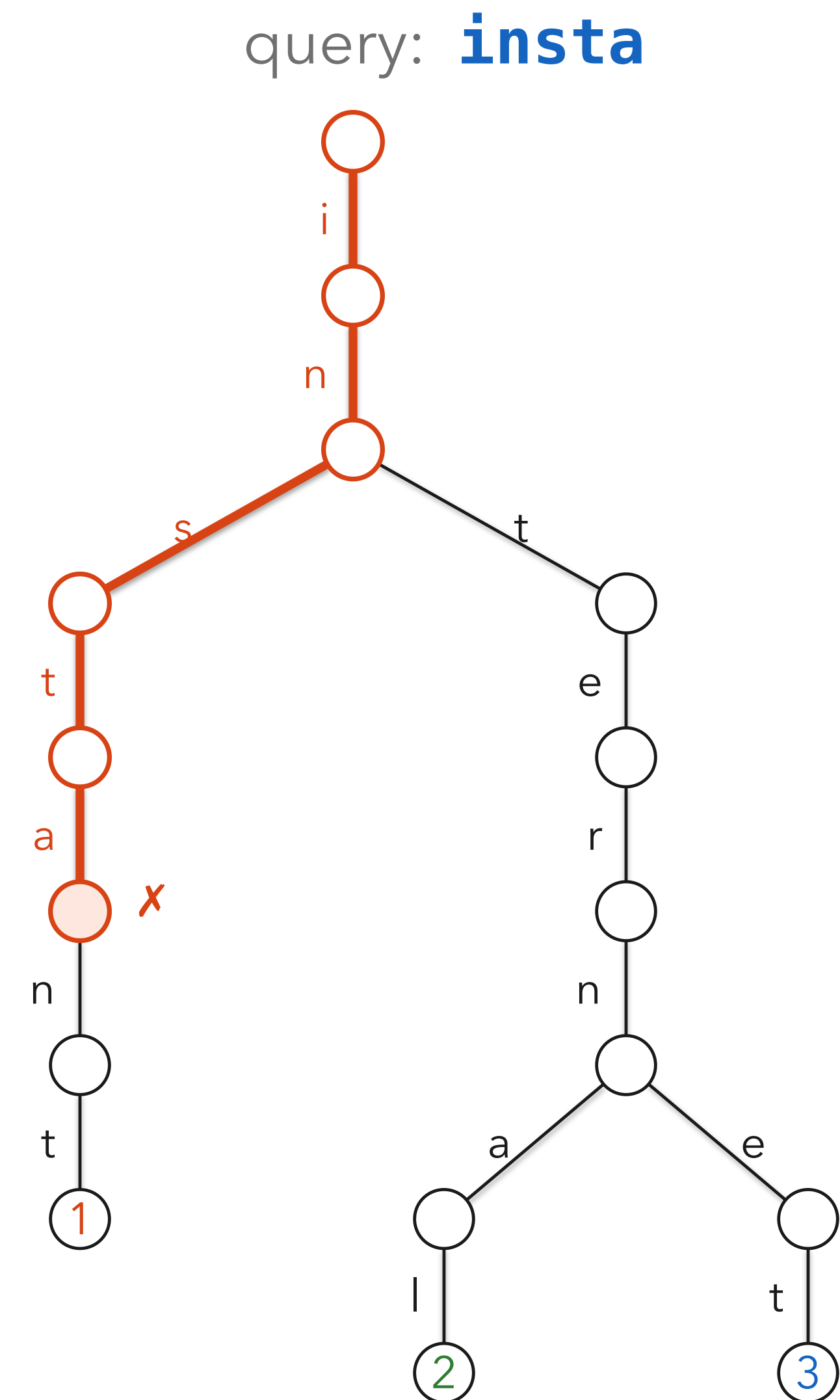
Tries: matching "insta"

Matching "insta"

Every character matches – but we land on a node with no value.

matched: **insta**

a node, but not a key:
"insta" is only a prefix



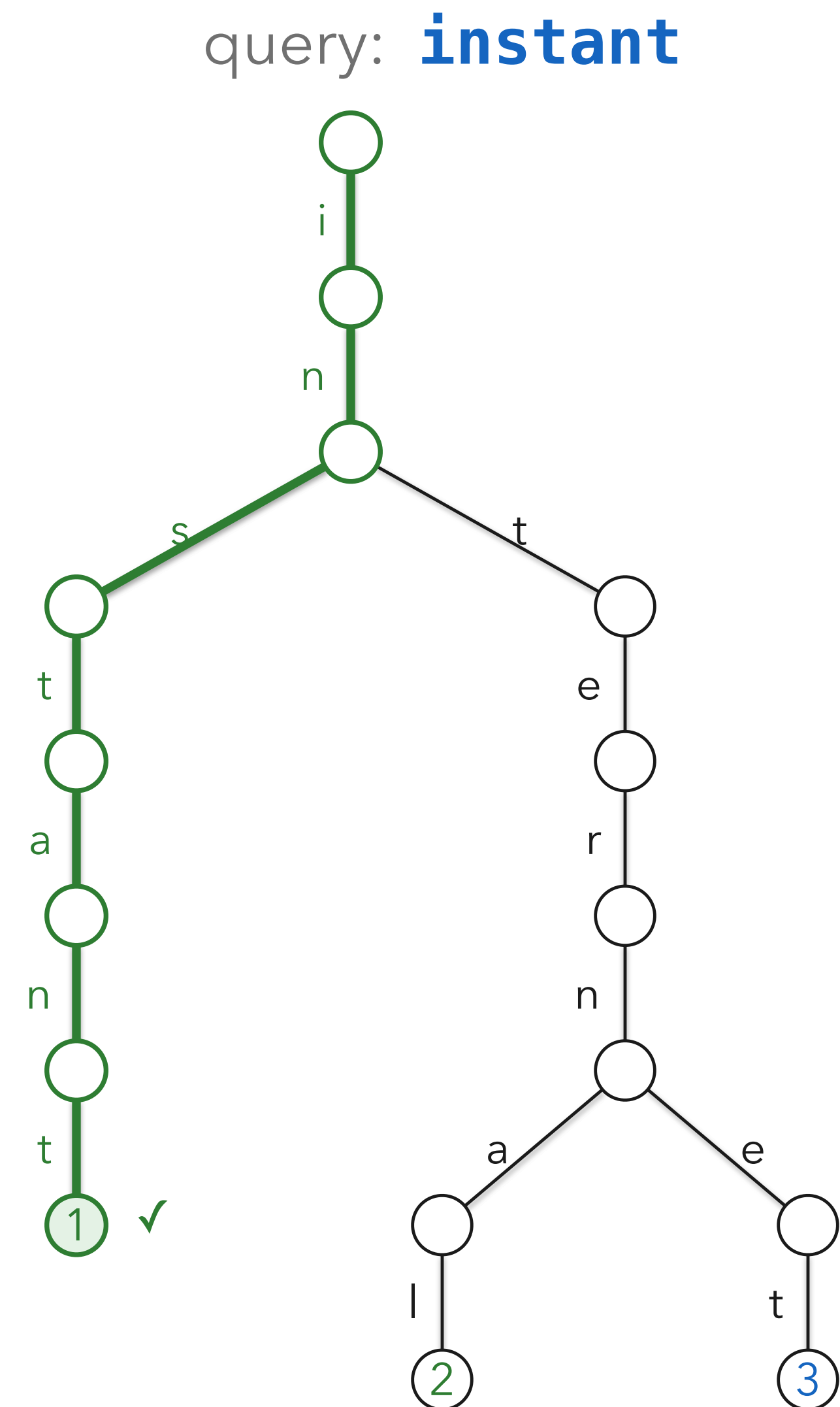
Tries: matching "instant"

Matching "instant"

Every character matches and the node we land on carries a value.

matched: **instant**

value 1 – it's a key



What does a trie cost?

- A lookup follows one edge per character of the query, so it takes **$O(n)$** steps – independent of how many keys the trie holds.
- If the keys have total length N , the trie has **$O(N)$** nodes.
- ...provided each step really is $O(1)$. That depends on how a node stores its outgoing edges:

representing a node's edges	cost per step	space per node
hash table from character to child	$O(1)$ expected	$O(\text{\#children})$
sorted list of (character, child)	$O(\log \Sigma)$	$O(\text{\#children})$
array of size $ \Sigma $	$O(1)$ worst case	$O(\Sigma)$ – wasteful for large Σ

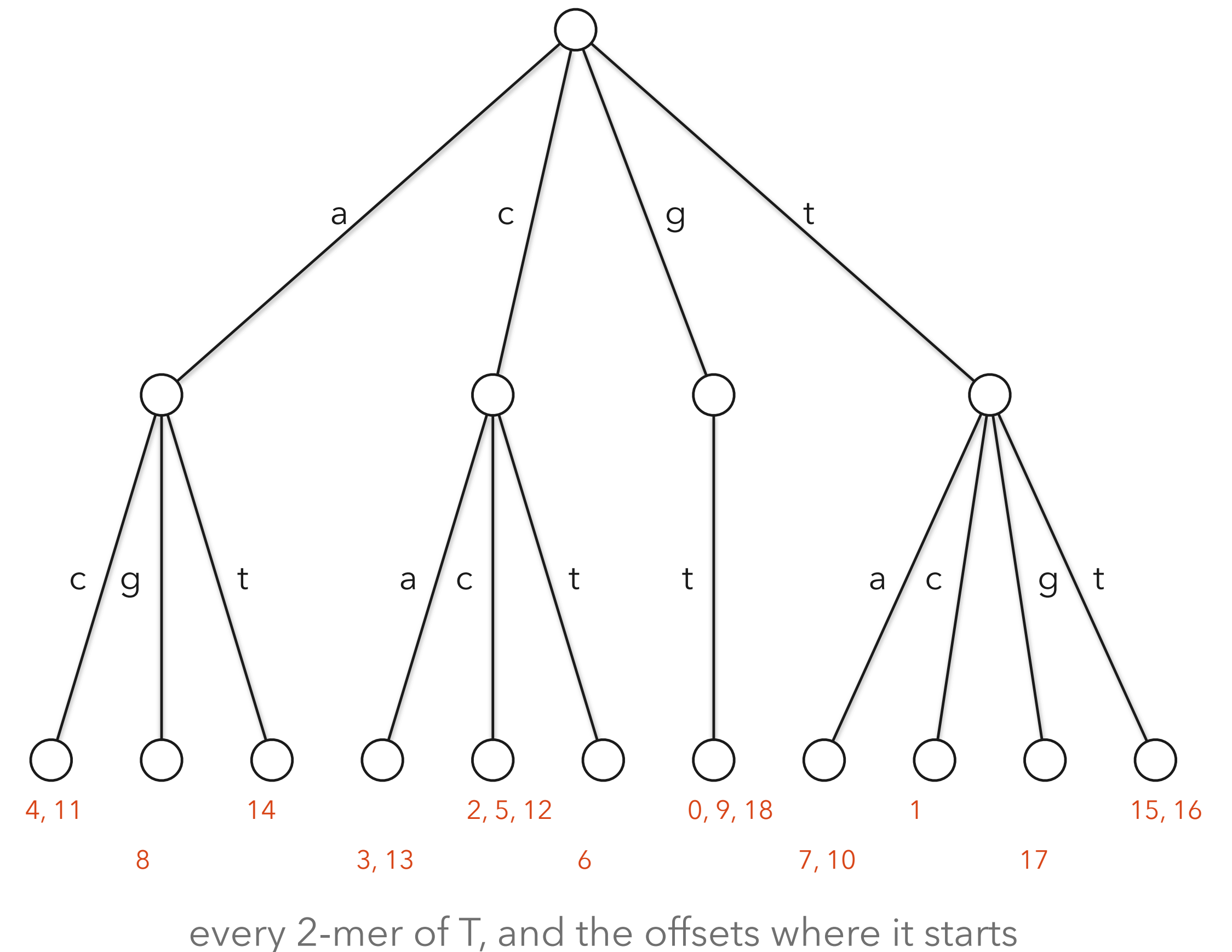
So “ $O(n)$ lookup” quietly assumes $|\Sigma|$ is a small constant. For DNA ($|\Sigma| = 4$) that is fair; for Unicode it is not.

A trie as an index over a text

T = **gtccacctagtagtaccatttgt**

- Take every substring of a fixed length – here every 2-mer – and record where it occurs.
- The trie maps each 2-mer to the **offsets** where it begins in T.
- A 2-mer that occurs more than once simply carries several offsets: **cc** → **2, 5, 12**.

This is an index over substrings of T.
Everything after this slide is an index over its
suffixes instead – and that one change buys
us every query length at once.



Tries: implementation

```
class TrieMap(object):
    """ Map from strings to values, stored as a trie. """
    def __init__(self, kvs):
        self.root = {}
        for k, v in kvs: self.add(k, v)

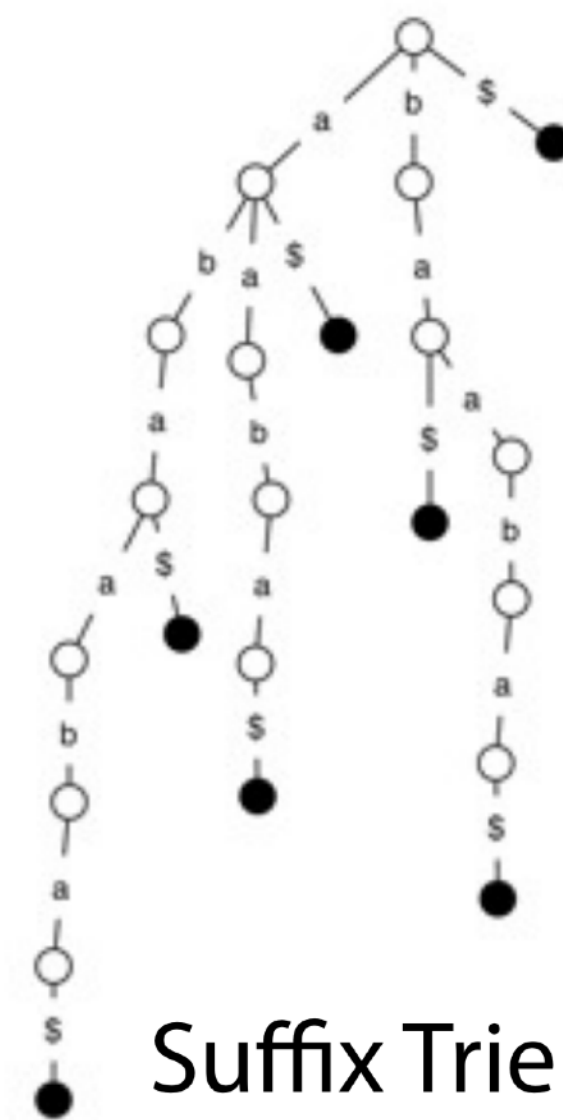
    def add(self, k, v):
        cur = self.root
        for c in k:
            # one edge per character
            if c not in cur: cur[c] = {} # make the edge
            cur = cur[c]
        cur['value'] = v # the path ends in a key

    def query(self, k):
        cur = self.root
        for c in k:
            if c not in cur: return None # fell off the trie
            cur = cur[c]
        return cur.get('value') # None if it is only a prefix
```

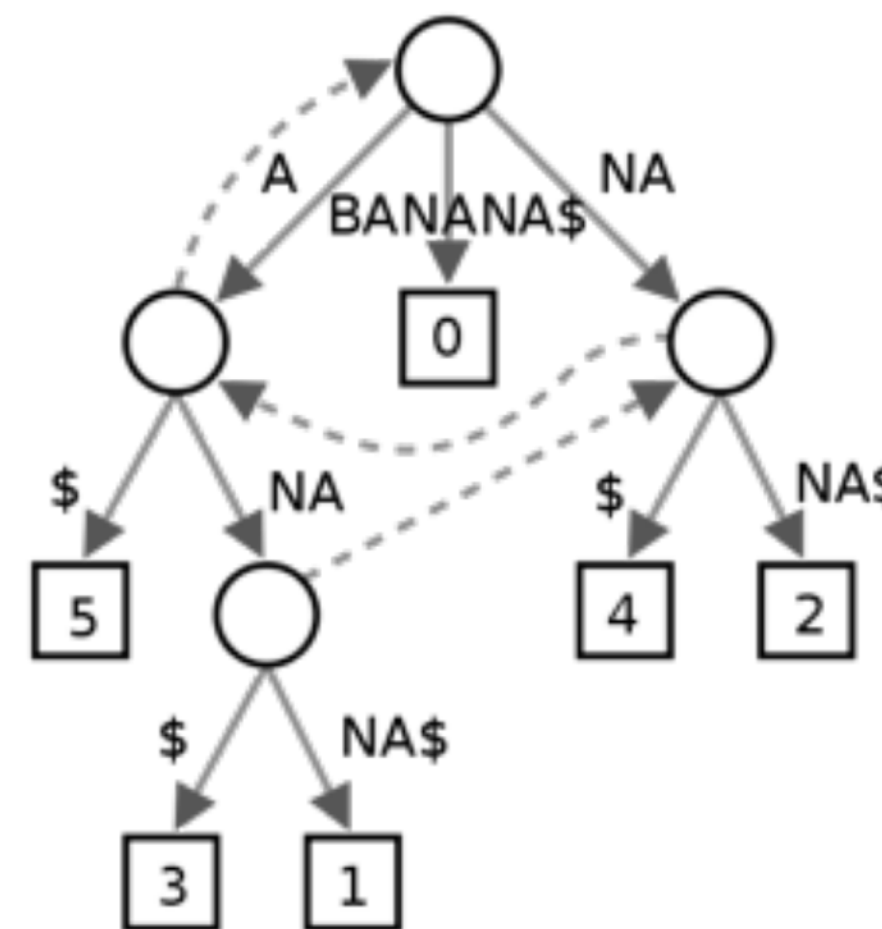
- A node is just a dict from character to child.
- **add** walks down, creating edges it does not find.
- **query** walks down and fails in exactly two ways: it falls off, or it lands on a node with no value.
- *Those two failure modes are what we cover next.*

Indexing with suffixes

- So far every index we have built works by extracting **substrings** of T – and we had to choose their length up front.
- A different idea: extract the **suffixes** of T . There are only m of them, and **every substring of T is a prefix of one of them.**
- That one observation leads to four data structures:



today



Suffix Tree

today

6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

Suffix Array

next lecture

\$BANANA
A\$BANAN
ANASBAN
ANANASB
BANANA\$
NASBANA
NANASBA

FM Index

next lecture

Suffix tries

A trie containing every suffix of T

Suffix trie: the idea

- Build a **trie** containing all **suffixes** of a text T.
- T has m suffixes, but writing them all out takes **$m(m+1)/2$** characters.
- The trie shares common prefixes, so it will be smaller than that – the question is *how much* smaller.

Hold on to this number. In about ten slides it becomes the reason suffix tries are unusable on a genome.

T = abaaba\$

abaaba\$

abaaba\$

baaba\$

aaba\$

aba\$

ba\$

a\$

\$

$m(m+1)/2 = 28$
characters

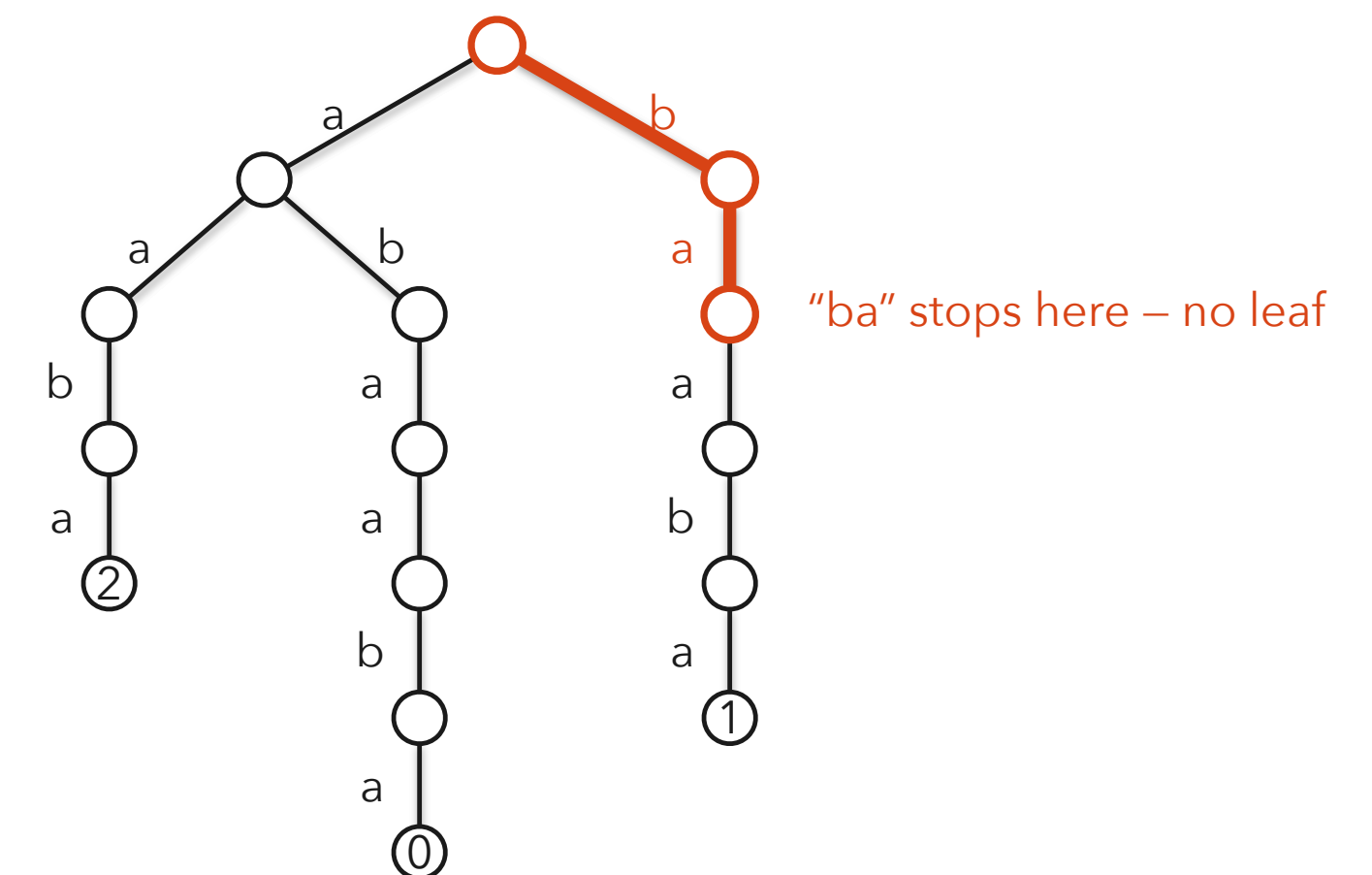
Why the terminator \$?

- \$ is a character that appears **nowhere else** in T , and sorts before every other character.
- It buys exactly one thing, and it is the thing we need:

no suffix of $T\$$ is a prefix of another suffix of $T\$$

- Without it, a short suffix can vanish inside a longer one. In **abaaba**, the suffix **ba** is a prefix of the suffix **baaba** – so “ba” ends in the middle of a path, not at a leaf.
- With \$, every suffix ends at a leaf, and **leaves correspond one-for-one with suffixes.**

suffix trie of abaaba – no \$



Building the suffix trie of abaaba\$

Step 1 of 3

inserted: abaaba\$

Insert the longest suffix first. Nothing to share yet
– it is a bare chain.

	now	when done
nodes	8	22
leaves	1	7



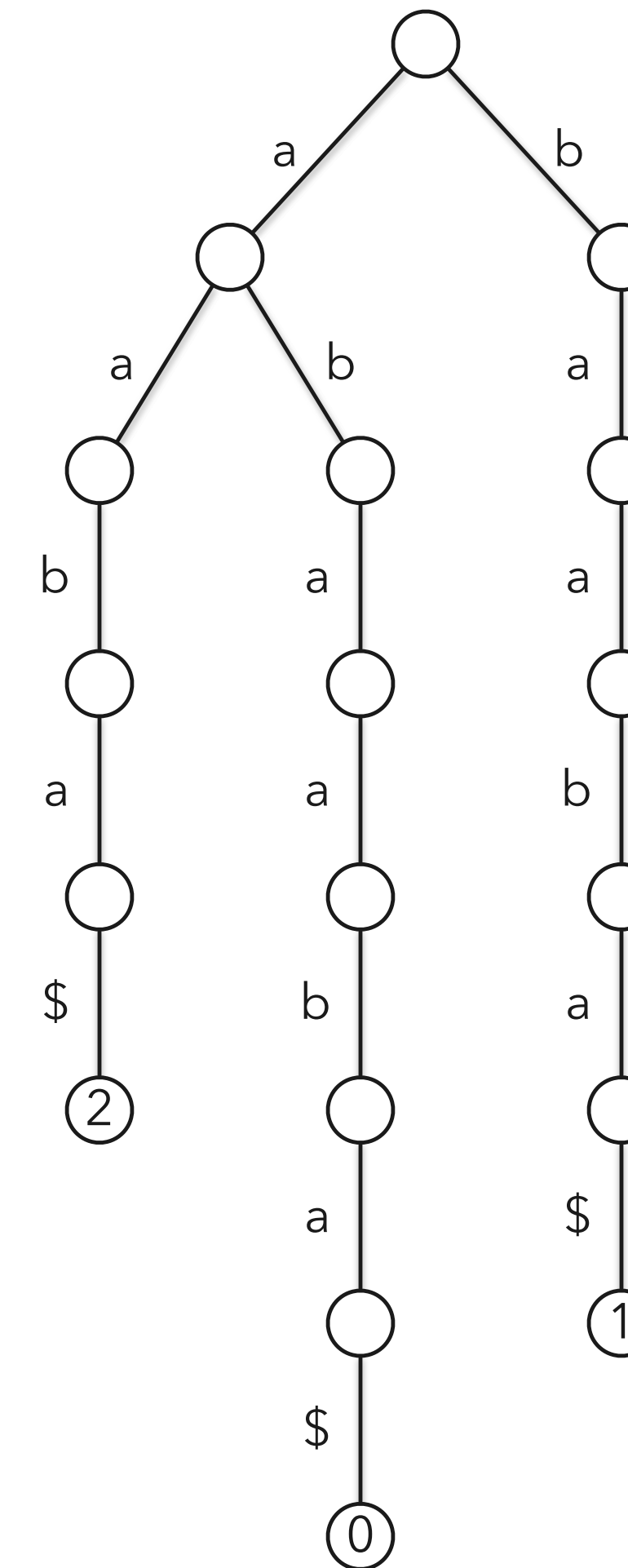
Building the suffix trie of abaaba\$

Step 2 of 3

inserted: **abaaba\$** **baaba\$** **aaba\$**

Each new suffix walks down as far as the trie already spells it, then branches off.

	now	when done
nodes	18	22
leaves	3	7



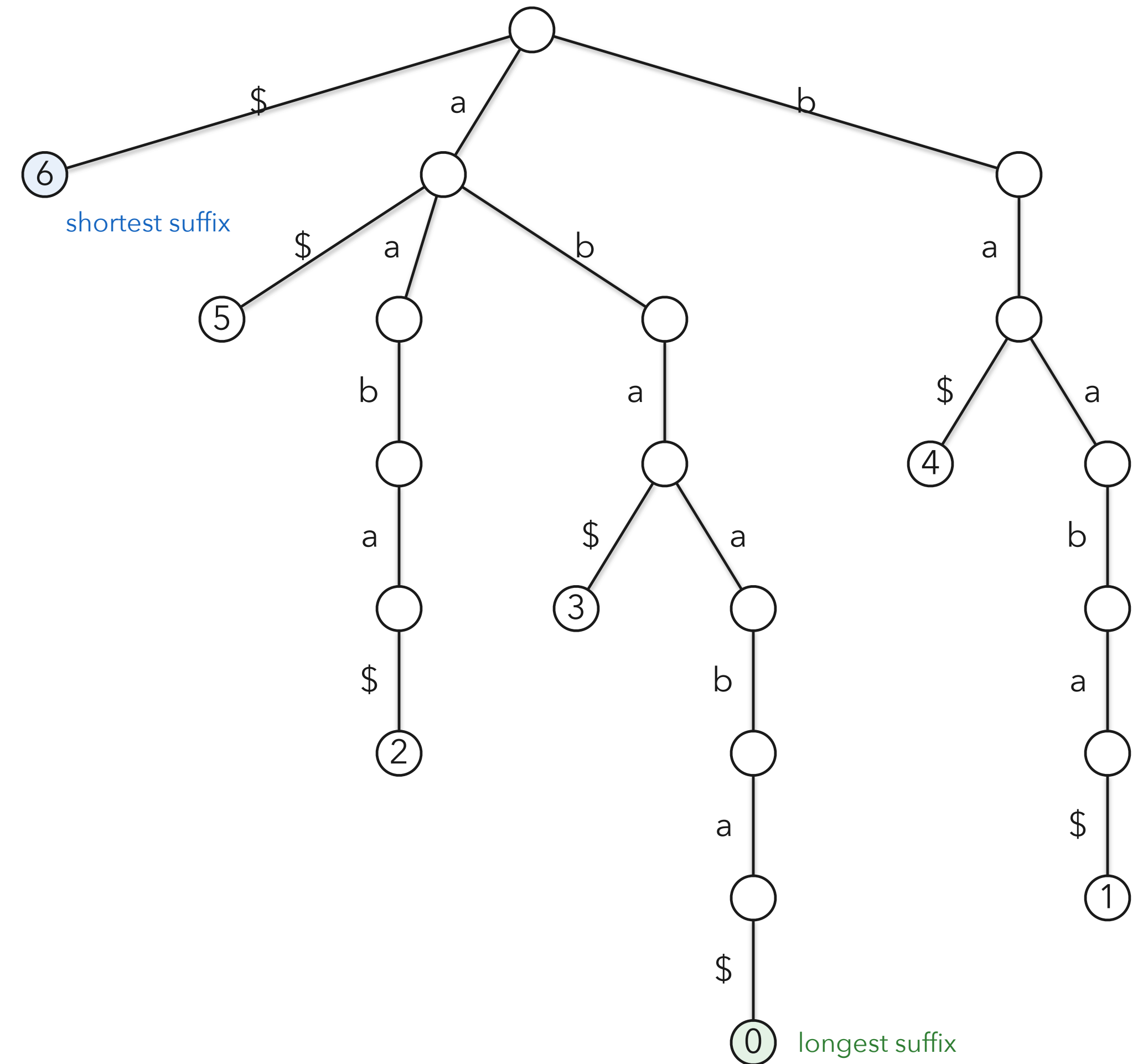
Building the suffix trie of abaaba\$

Step 3 of 3

inserted: abaaba\$ baaba\$ aaba\$ aba\$ ba\$
a\$ \$

All 7 suffixes inserted. Every path from the root to a leaf spells one suffix.

	now	when done
nodes	22	22
leaves	7	7



What the nodes are

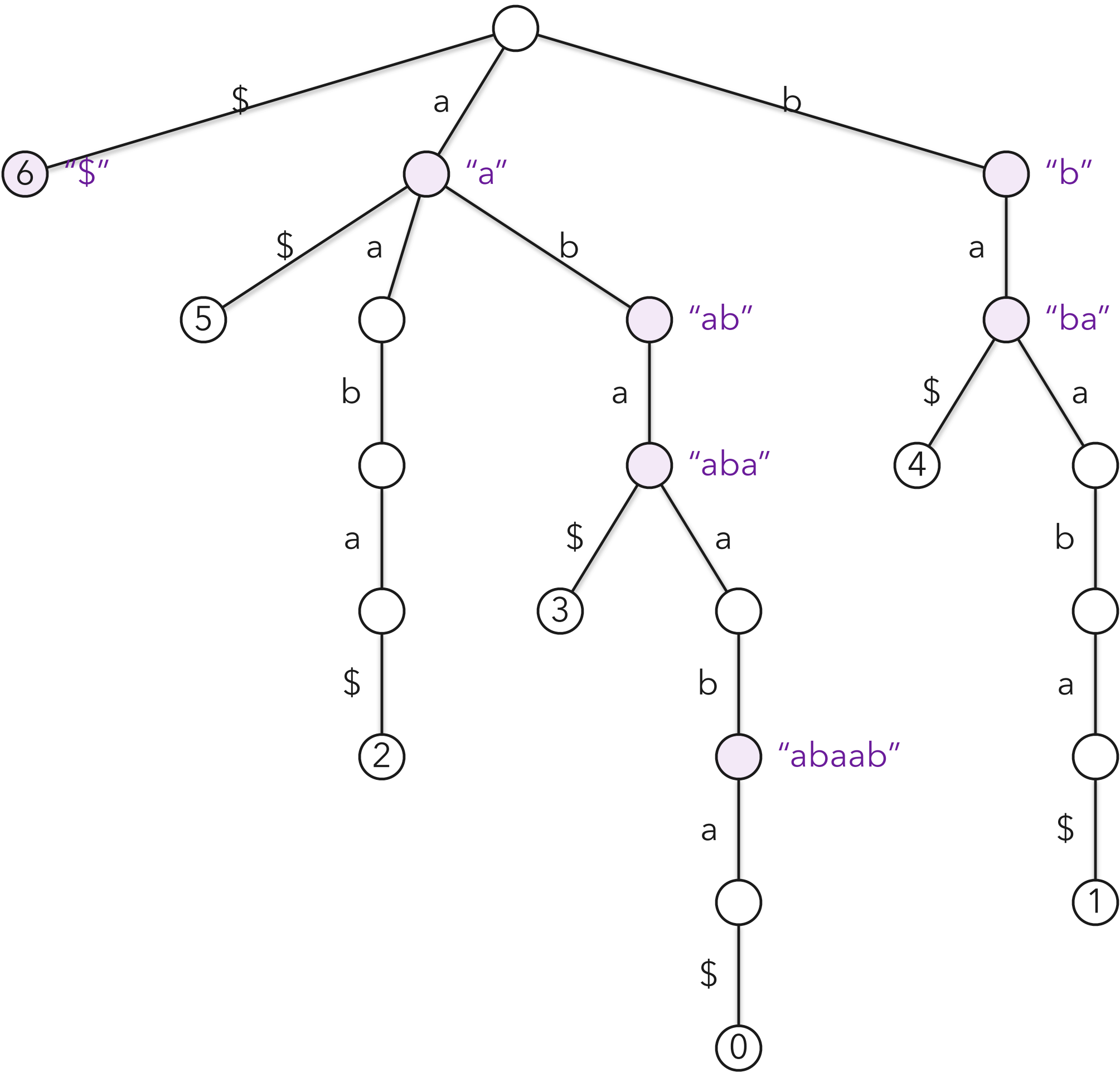
Every node of the suffix trie is one distinct substring of T.

- A node's **label** is the string spelled out on the path from the root down to it.
- Every substring of T is a prefix of some suffix of T, so it is spelled by some root-to-node path – and distinct substrings give distinct paths.

#nodes = #distinct substrings + 1

T	distinct substrings	suffix trie nodes
abaaba\$	21	22

Three later results are this sentence in disguise: is S a substring, how often, and how big can the trie get?



Suffix trie: is S a substring of T?

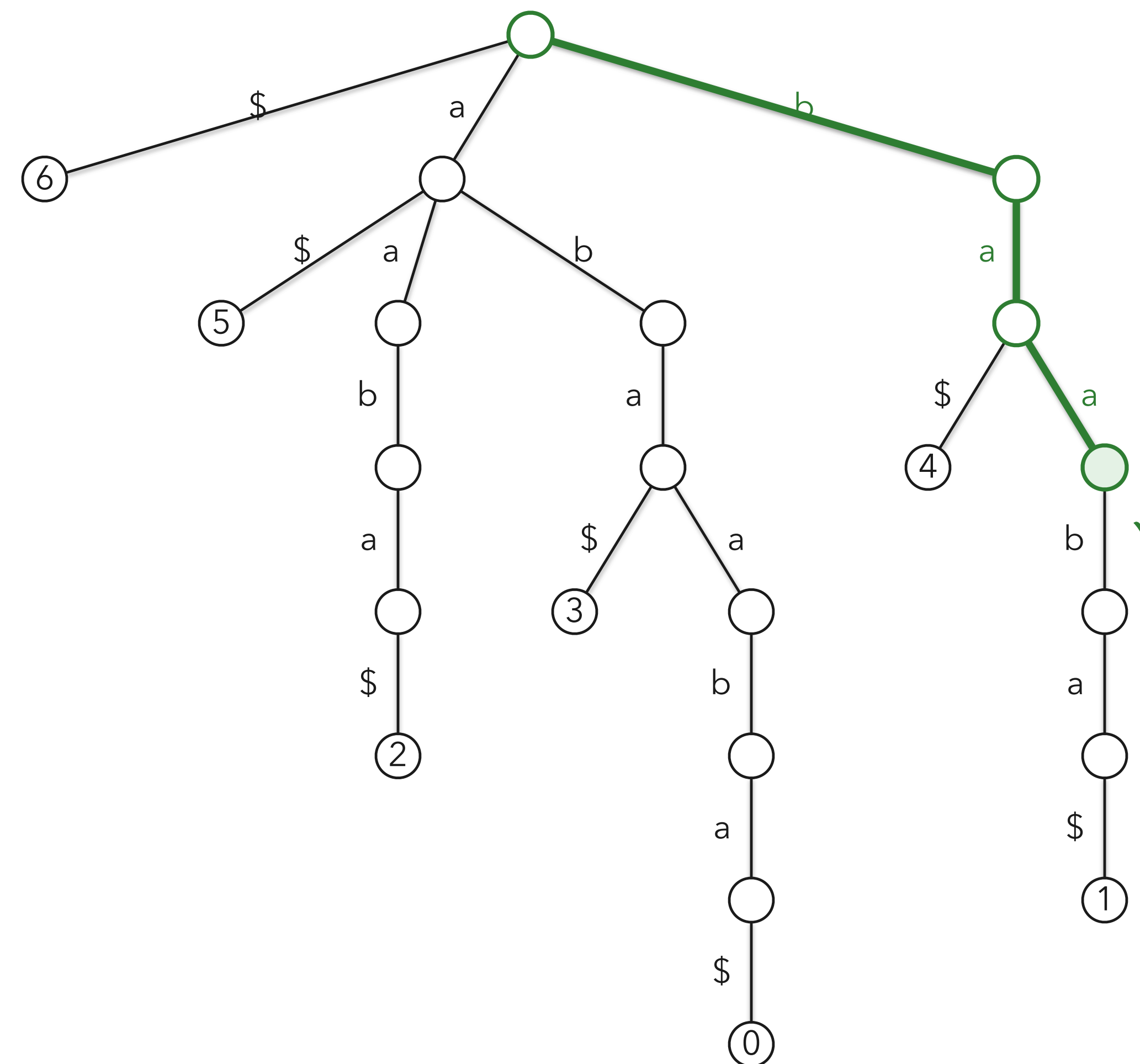
T = abaaba\$

Start at the root and follow the edges labelled with the characters of S. If you never fall off, S is a substring.

- Every substring of T is a prefix of some suffix of T.
- Falling off – no outgoing edge for the next character
– is the only way to fail.

S = baa

yes – every character matched



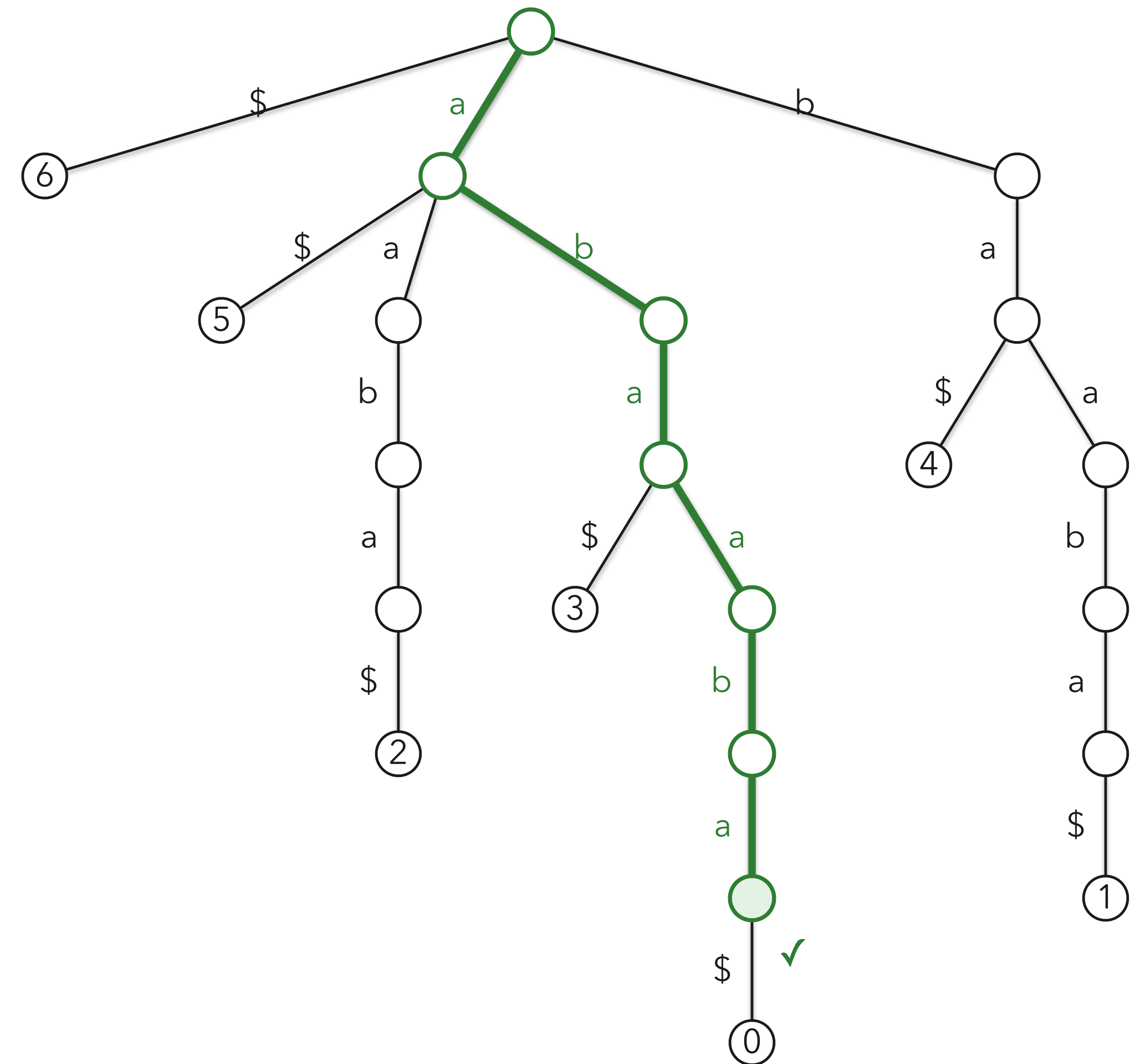
Suffix trie: is S a substring of T?

The same walk, with a longer query. S is all of T except the $\$$, so the walk stops one edge short of a leaf.

S = abaaba

yes – S is T itself, minus the \$

T = abaaba\$



Suffix trie: is S a substring of T?

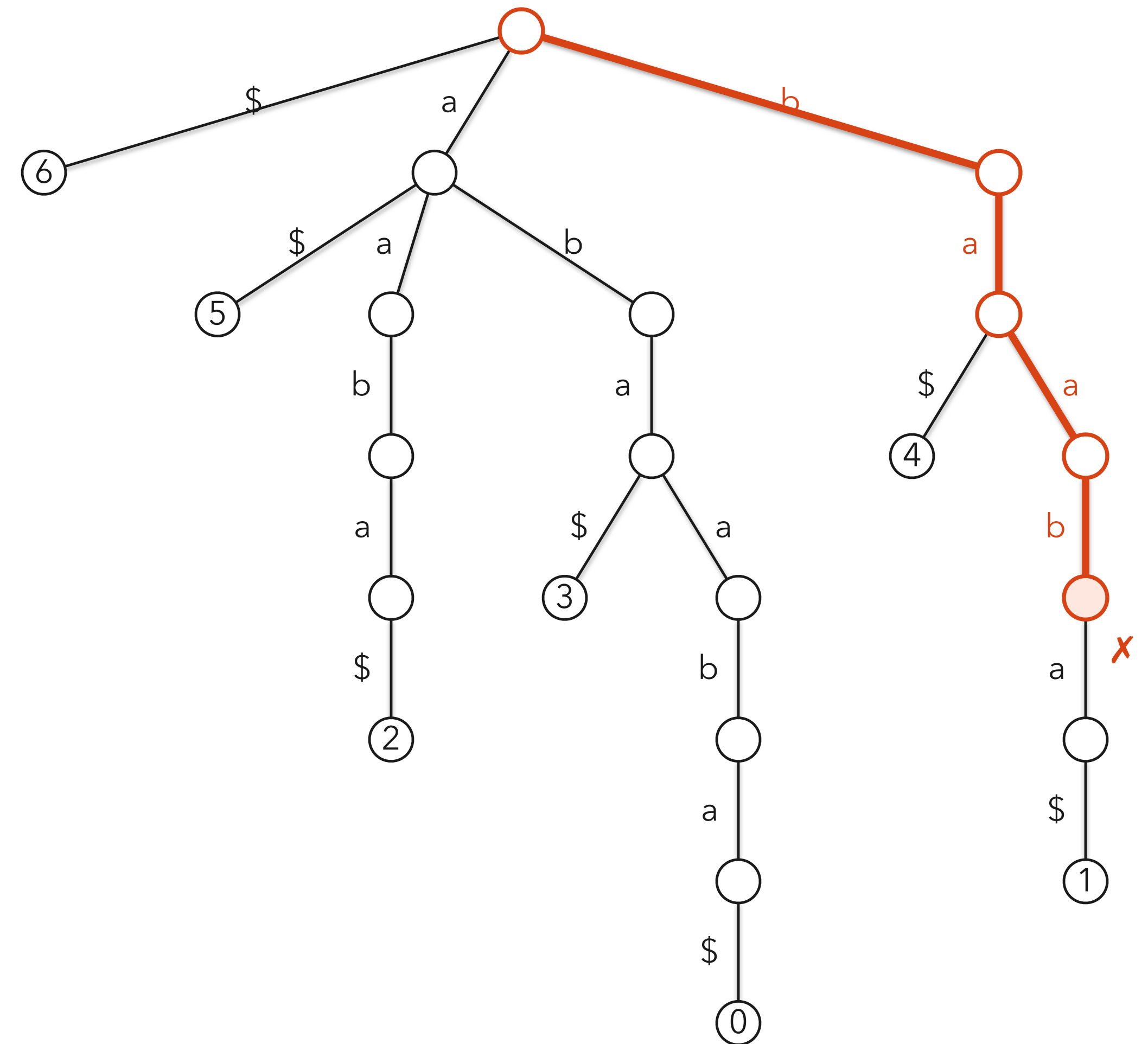
T = abaaba\$

Here the walk dies. After "baab" there is no outgoing edge labelled "b".

S = baabb

no – we fell off after "baab"

The prefix "baab" is a substring of T; "baabb" is not. The trie tells us both, and tells us exactly where the query stopped matching.



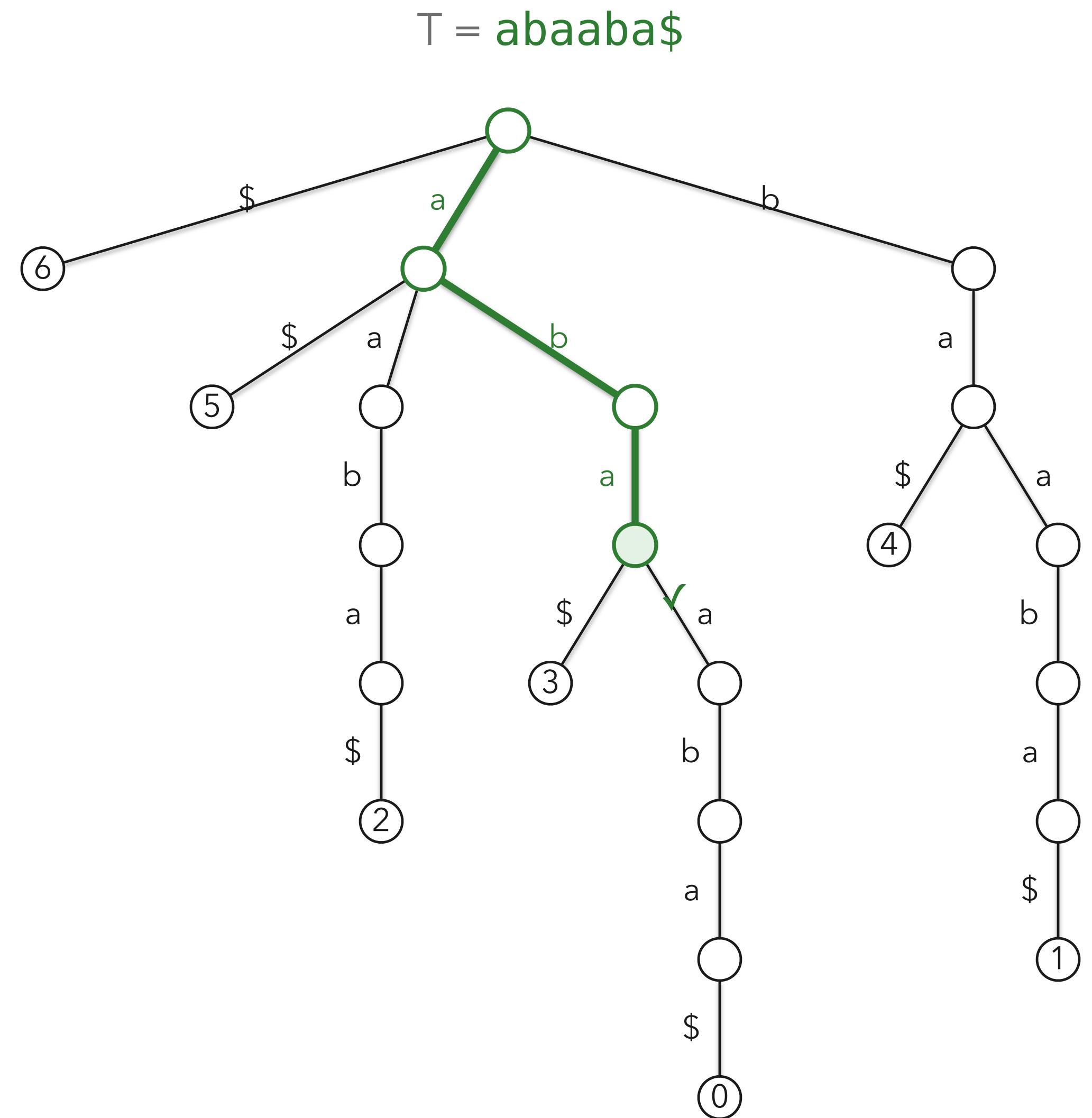
Suffix trie: is S a suffix of T?

Same walk as before, plus one extra check at the end: does the node we land on have a \$ child?

- A \$ child means some suffix ends right here.
- “baa” would fail this test: it is a substring of T, but not a suffix.

$S = aba$

yes – node(“aba”) has a \$ child



Suffix trie: how many times does S occur?

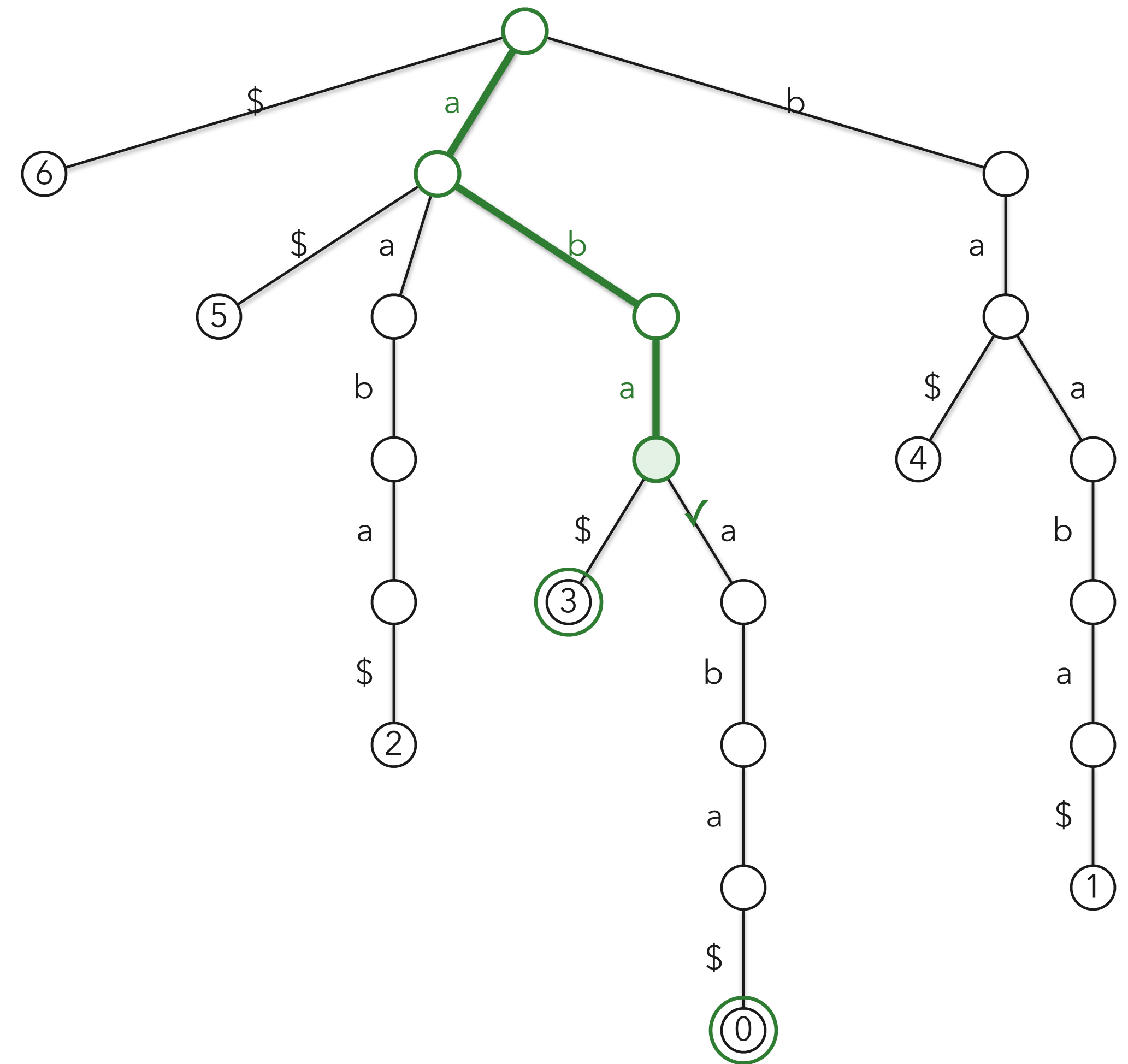
T = abaaba\$

Walk to node(S), then count the leaves in the subtree below it. Each leaf is one suffix that starts with S – that is, one occurrence.

- If the walk falls off, the answer is 0.
- Leaves can be counted with a depth-first traversal – $O(\text{size of the subtree})$.

S = aba

2 occurrences (at offsets 0 and 3)

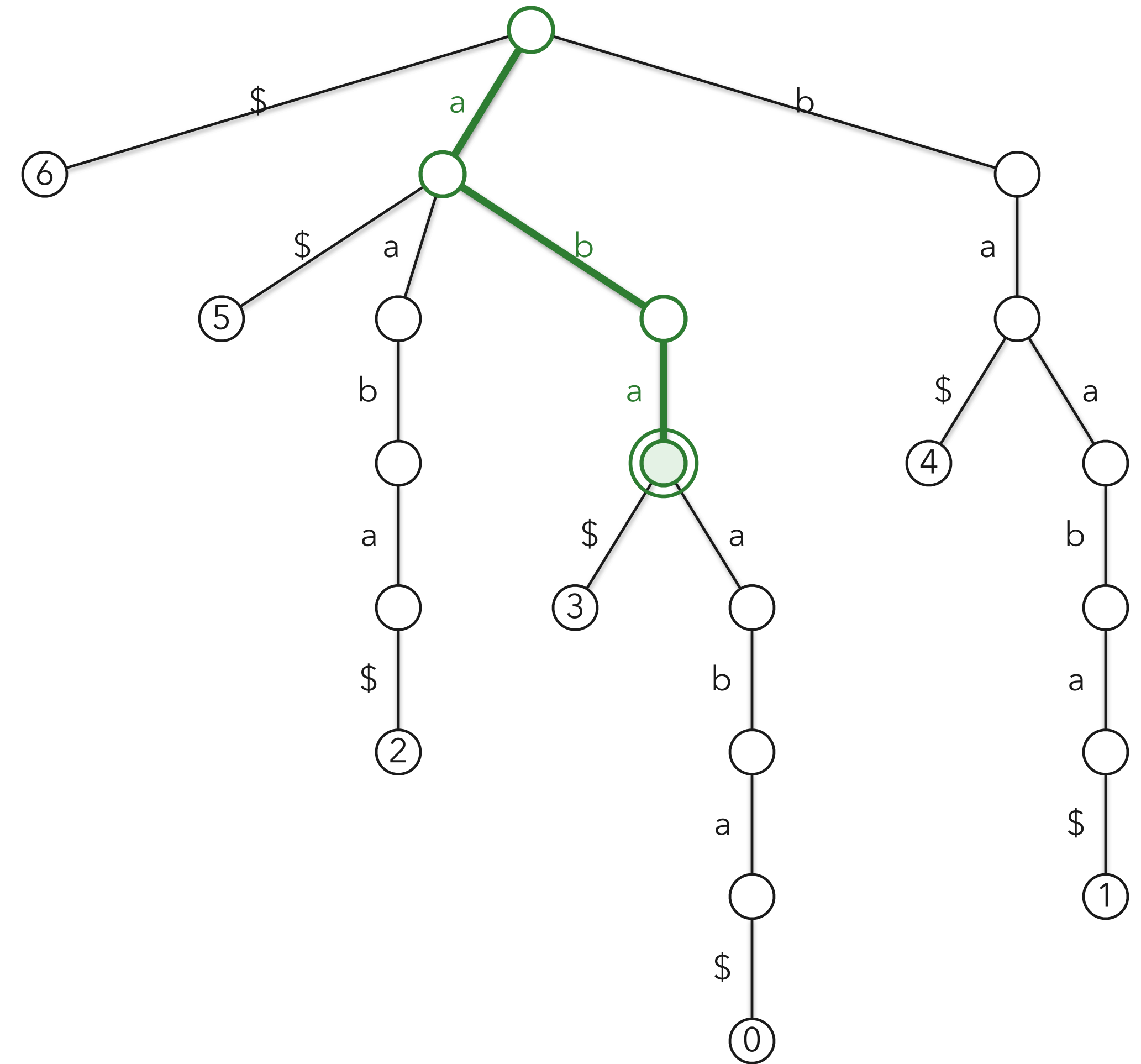


Suffix trie: longest repeated substring

If a node has two or more children, its string occurs at least twice – two different characters follow it in T. And the longest repeat must be such a node: if only one character ever followed it, adding that character would give a longer repeat.

longest repeated substring of abaaba\$
= "aba" (3 characters)

- In a suffix trie, depth is also the length of the substring.
- The converse fails: "ab" repeats too, but has one child – every "ab" in T is followed by "a". Such a repeat sits inside a longer one.



the ringed node is the deepest one with two children

Suffix trie: what it can do

query	how	time
is S a substring of T?	walk from the root; don't fall off	$O(n)$
is S a suffix of T?	...and check for a \$ child at the end	$O(n)$
where does S occur?	collect the leaf offsets below node(S)	$O(n + k)$
how often does S occur?	count the leaves below node(S)	$O(n + k)$
longest repeated substring	deepest node with ≥ 2 children	$O(\text{\#nodes})$

The first four are linear in the query, not in the text. The last one walks the whole structure – so its cost is whatever the trie's size turns out to be.

So: how big is this thing?

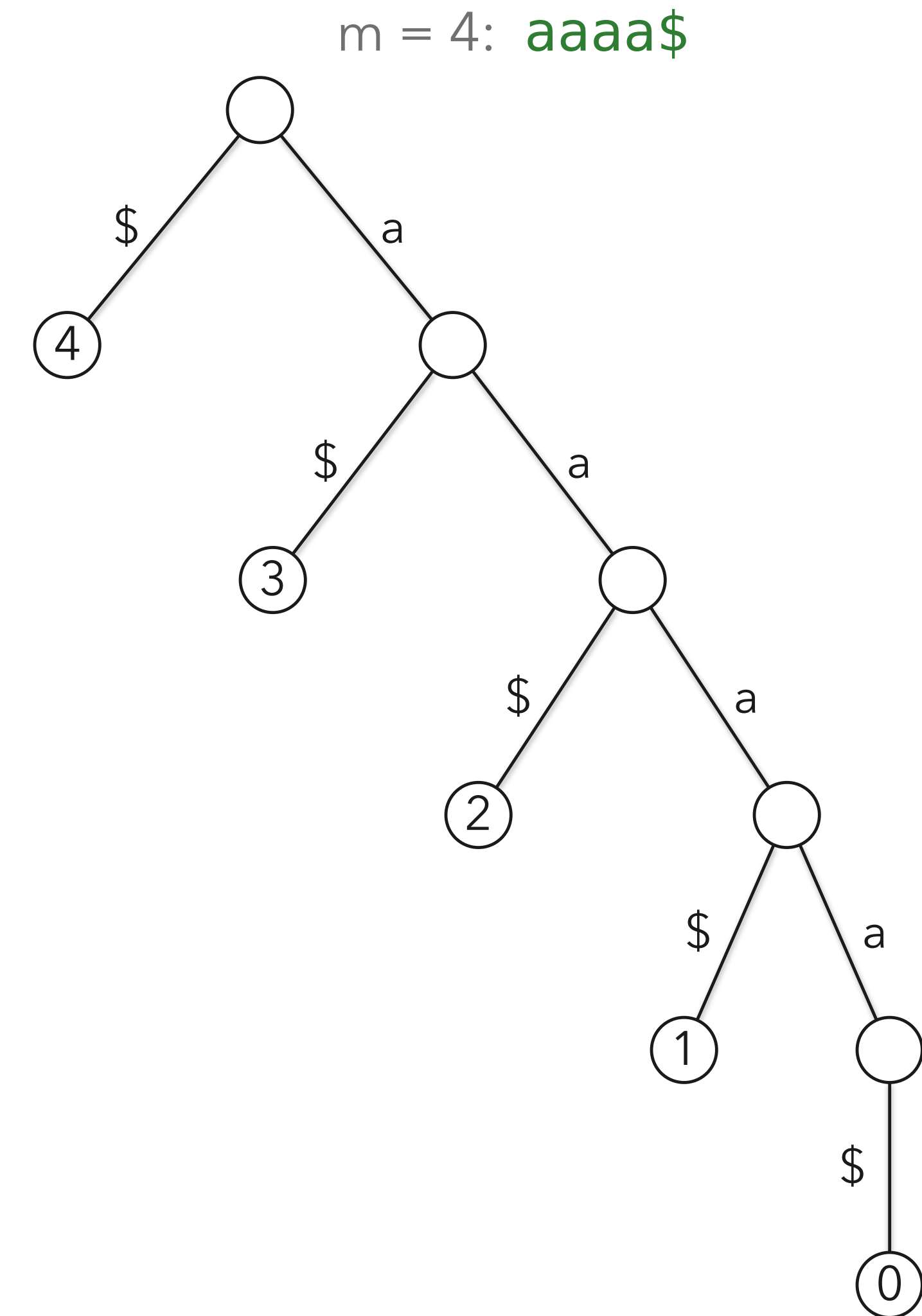
How big? The best case

$T = a^m\$$ – m copies of one character, then the terminator

- Every suffix is a prefix of every longer one, so the trie is a single chain with one \$ hanging off each node.
- 1 root
- m nodes with an incoming a edge
- $m + 1$ nodes with an incoming $\$$ edge

$2m + 2$ nodes – linear in m

Checked: $m = 4$ gives 10 nodes.



How big? The bad case

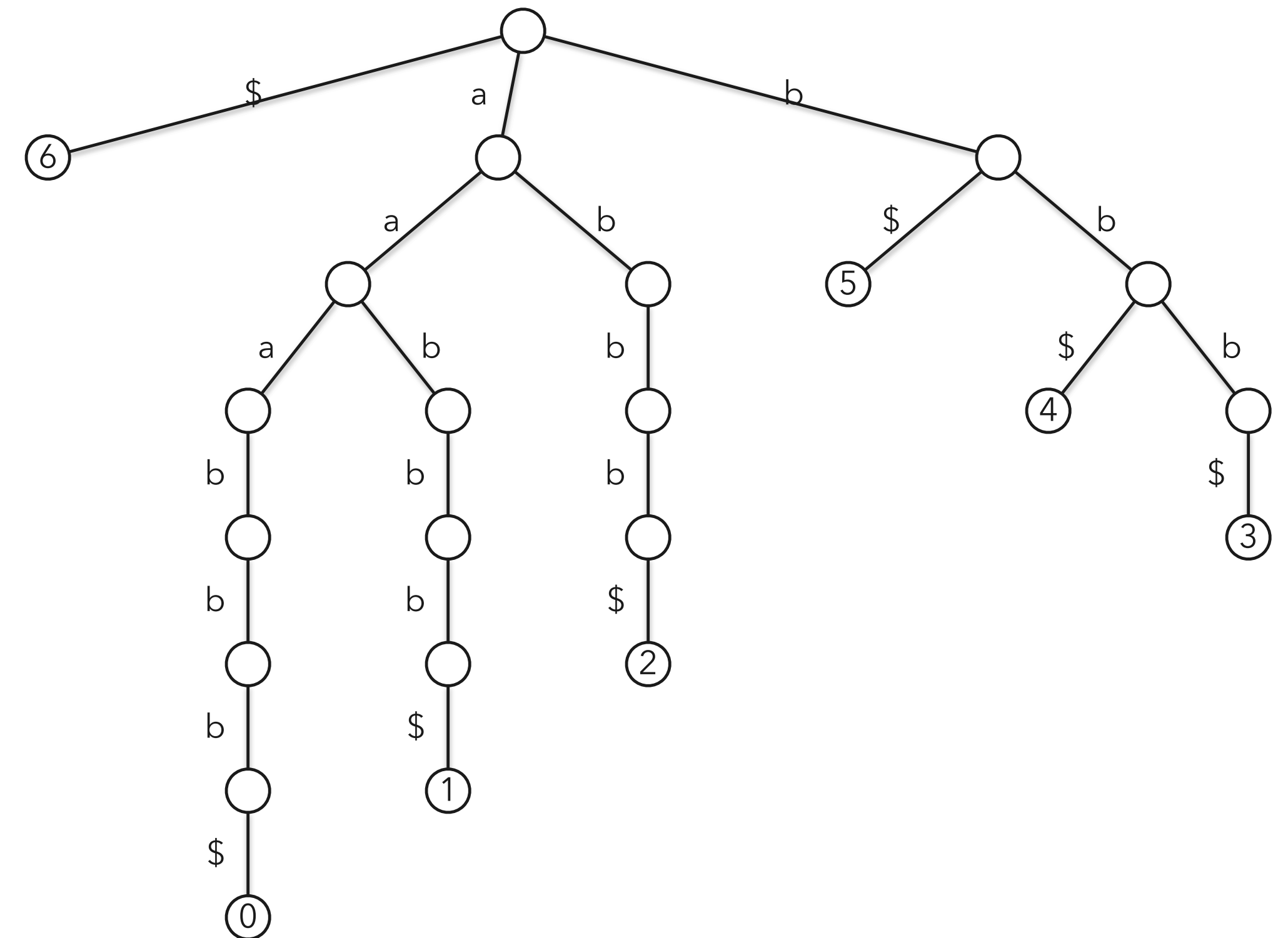
$T = a_n b_n \$$ – exponent $n = 3$, so $m = 2n = 6$ characters before the $\$$

- 1 root
- n nodes along the "b" chain
- n nodes along the "a" chain
- **n chains of n "b" nodes, one hanging off each "a" node**
- $2n + 1$ leaves below \$ edges

$n^2 + 4n + 2$ nodes, with $m = 2n$
 $\rightarrow$ quadratic in m

Checked: $n = 3$ gives 23 nodes ($3^2 + 4 \cdot 3 + 2 = 23$).

suffix trie of **aaabbbb\$**



How big? The upper bound

Could it be worse than quadratic?

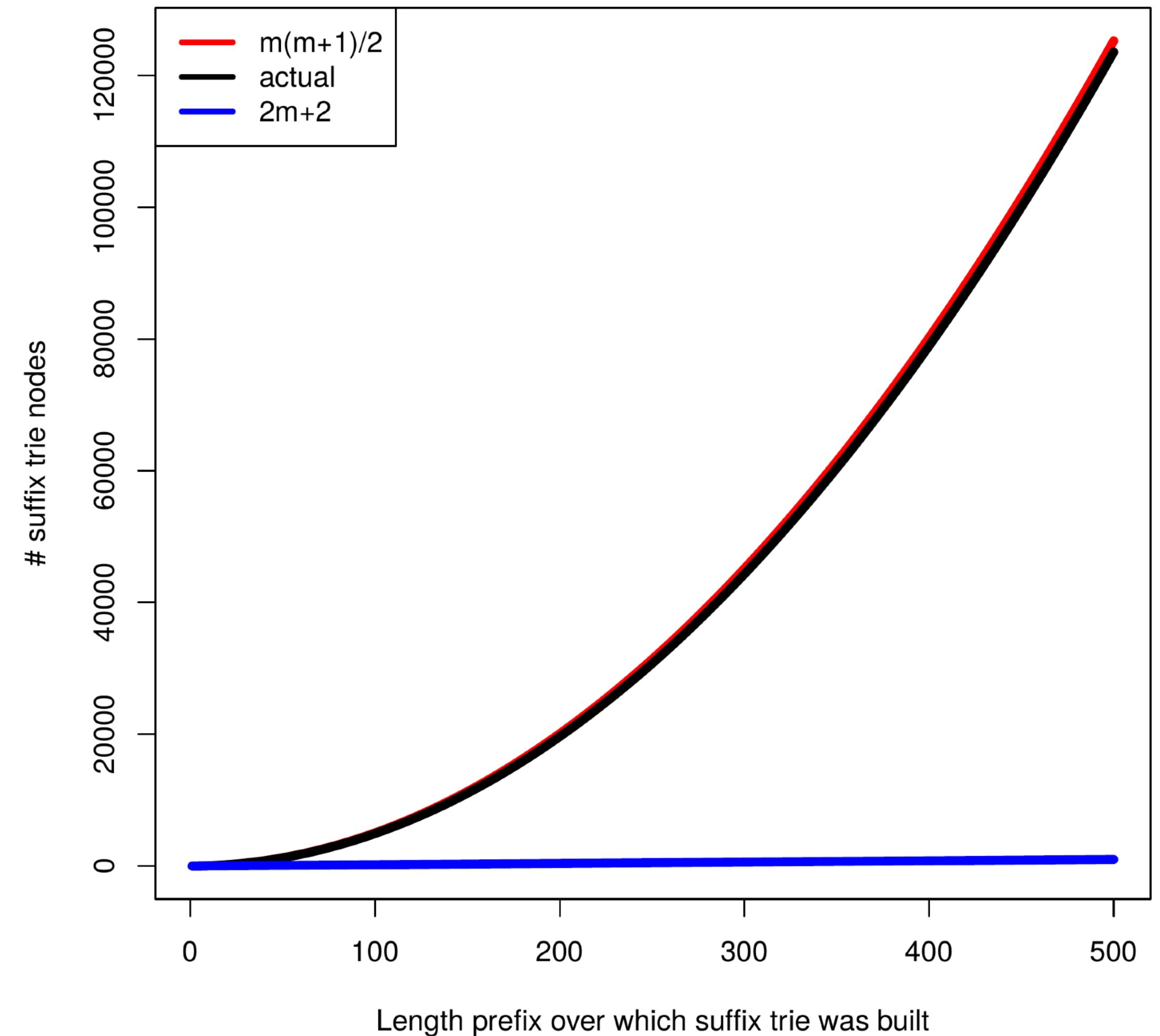
- **From the theorem: #nodes = #distinct substrings of T + 1.**
- T has at most m substrings of each length ℓ – one starting at each position – and lengths run from 1 to m.
- Counting them by starting position instead: **$m + (m-1) + \dots + 1 = m(m+1)/2$.**

$$\text{\#nodes} \leq 1 + m(m+1)/2 = O(m^2)$$

- ...and that is the same $m(m+1)/2$ we counted when we wrote the suffixes out longhand. Sharing prefixes did not change the order of growth.
- $a^n b^n$ shows the bound is tight up to a constant, so $O(m^2)$ is not merely an over-estimate.

How big? In practice

- Suffix tries built over the first 500 prefixes of the λ -phage genome.
- The black curve is the actual node count; red is the worst case $m(m+1)/2$; blue is the best case $2m+2$.



How big? In practice

- Suffix tries built over the first 500 prefixes of the λ -phage genome.
- The black curve is the actual node count; red is the worst case $m(m+1)/2$; blue is the best case $2m+2$.

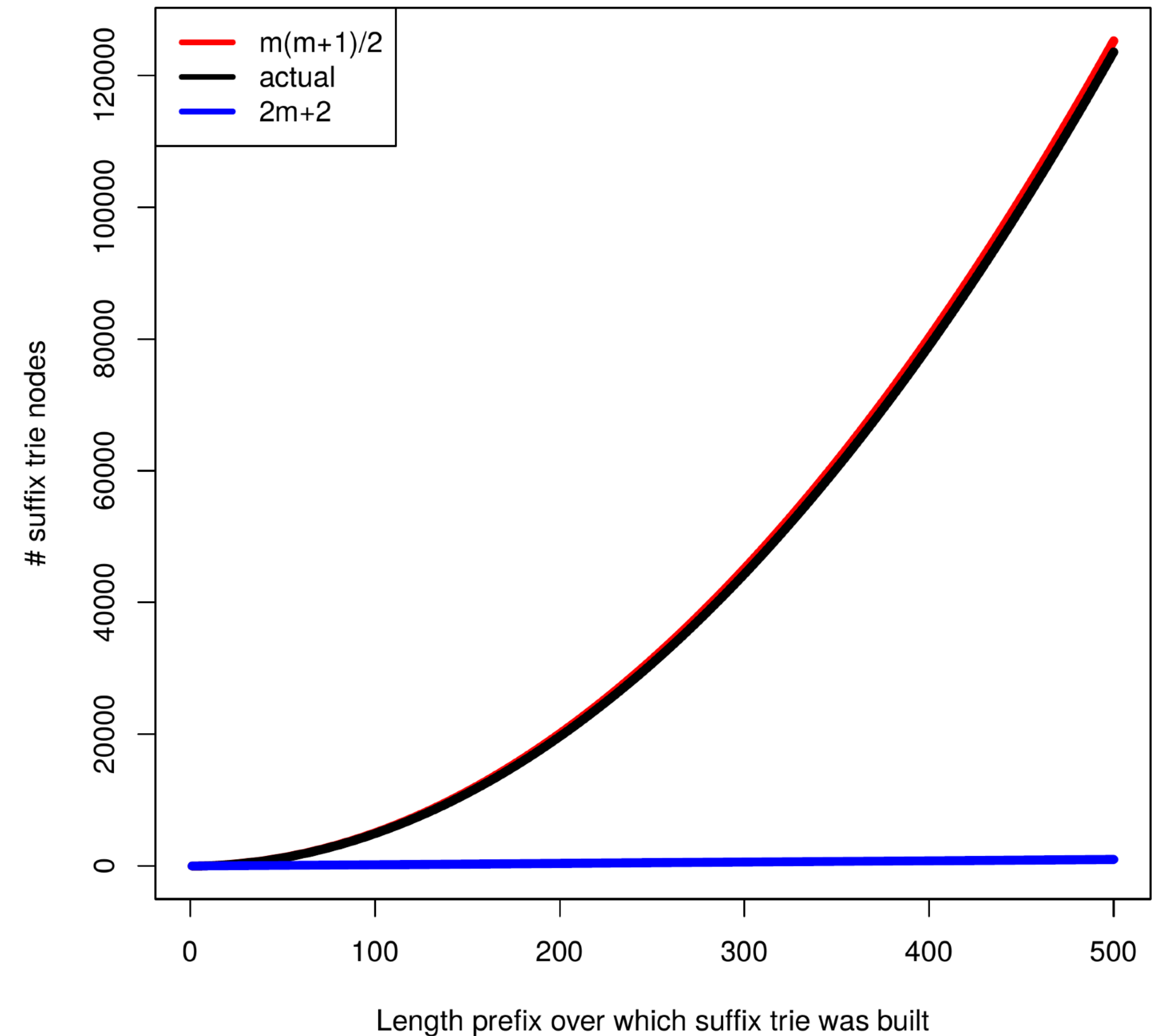
At $m = 500$: 123,000 nodes.

Worst case would be 125,250.

Real text tracks the worst case, not the best.

Extrapolating that rate to human chromosome 1 (250,000,000 bases): about 31×10^{15} nodes. At even one byte each, that is tens of petabytes.

The suffix trie is the right idea and the wrong data structure.



Suffix trees

The same information, with the chains squeezed out

Suffix tree: the definition

A Σ^+ -tree is a rooted tree whose edges are labelled with non-empty strings, where no node has two outgoing edges whose labels start with the same character.

It is compact if every internal node has at least 2 children.

- **words(T)** = every string spelled out by a path from the root; the path may stop mid-edge.
- The **suffix tree** of s is a compact Σ^+ -tree with $\text{words}(T) = \{ s' : s' \text{ is a substring of } s \}$.

	suffix trie of abaaba\$	suffix tree of abaaba\$
edge labels	one character	a non-empty string
internal nodes	may have 1 child	must have ≥ 2 children
total nodes	22	11
leaves	7	7

Two kinds of depth

suffix tree of **abaaba\$**

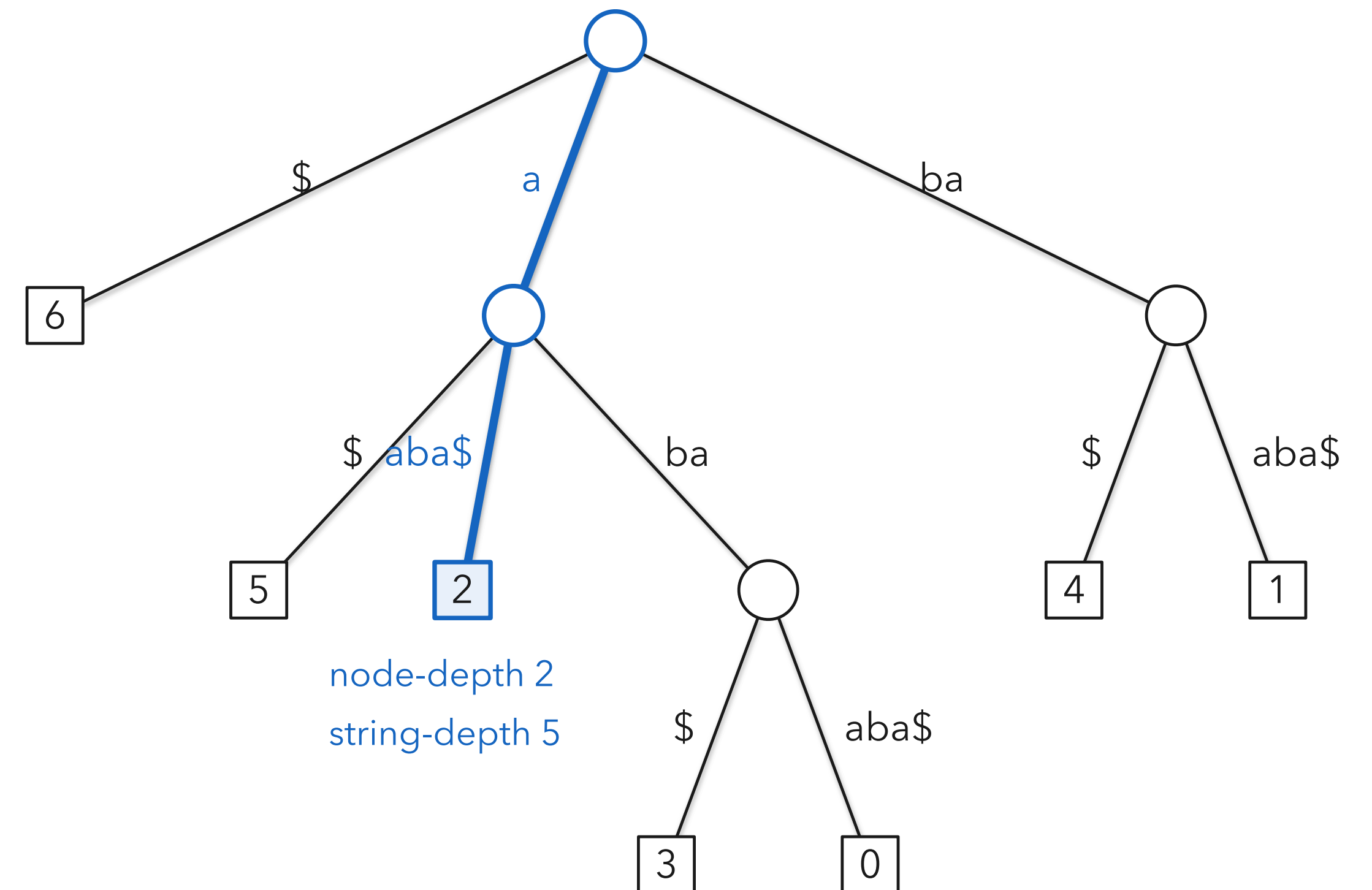
- **node-depth(v)** – how many edges you follow from the root to reach v.
- **string-depth(v)** – the total length of those edge labels, i.e. $|\text{string}(v)|$.
- In a suffix trie every edge is one character, so the two agree. In a suffix tree they come apart, and it is almost always **string-depth** that we mean.

for the highlighted node:

$\text{string}(v) = \text{"aaba\$"}$

$\text{node-depth}(v) = 2$

$\text{string-depth}(v) = 5$

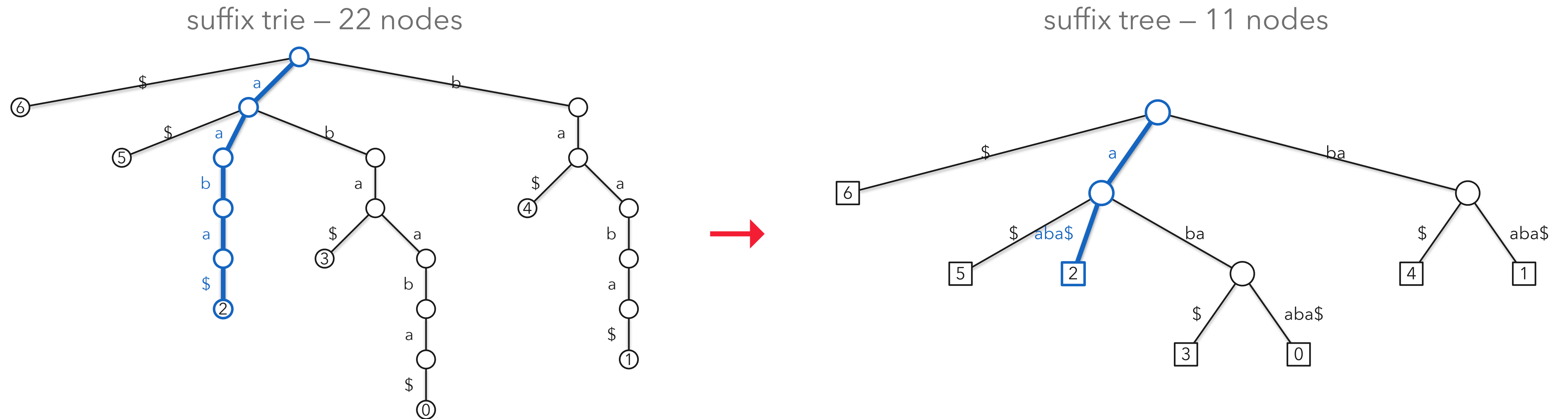


node-depth 2
string-depth 5

string-depth(v) ≥ node-depth(v), always.

Idea 1: coalesce non-branching paths

A node with exactly one child carries no information – there was never a choice to make there. Replace the chain with one edge, labelled by the string it spelled.

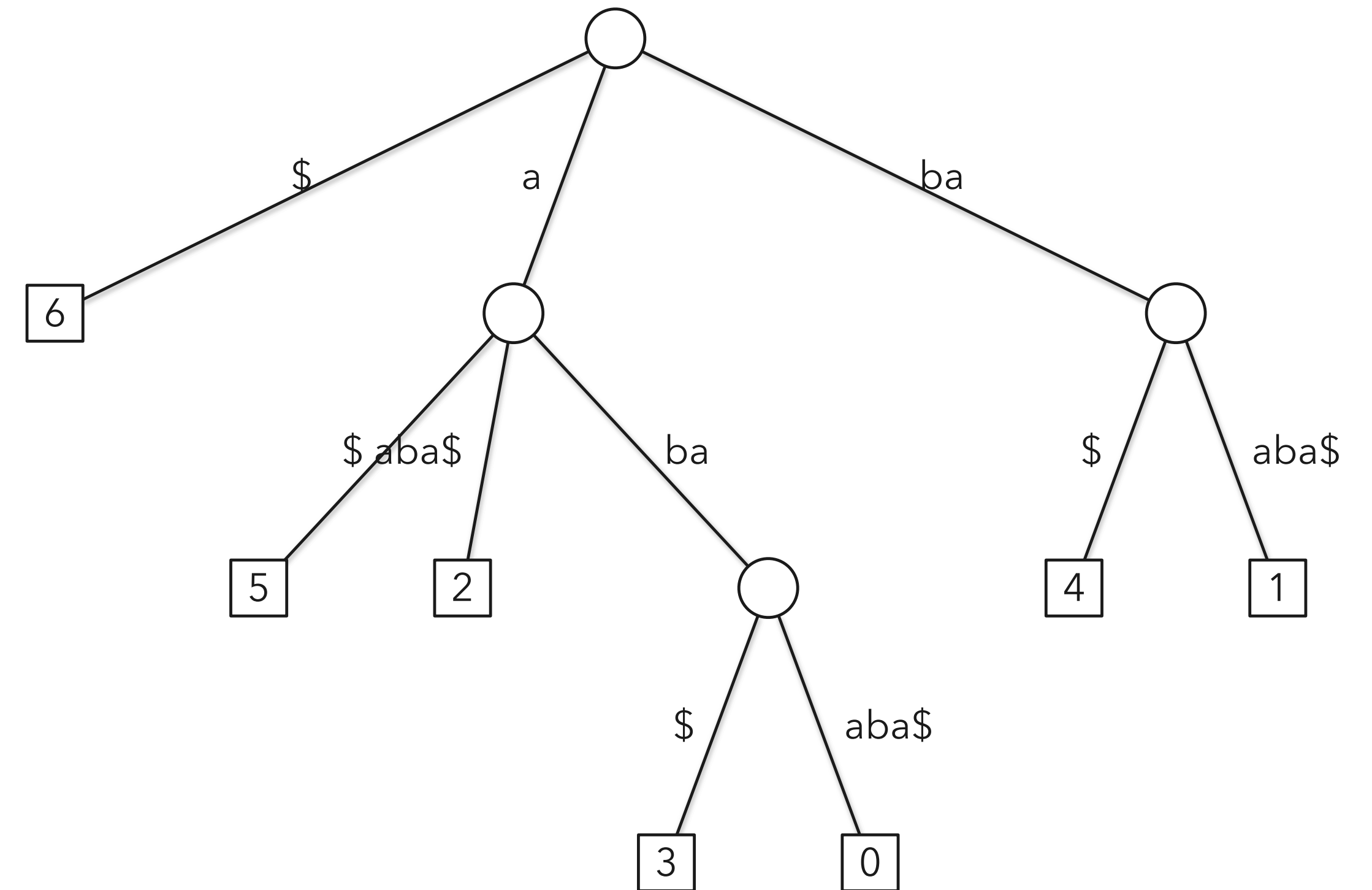


Below node "a", four blue edges on the left spell "aba\$" one character at a time. On the right they are a single edge with that label – same information, three fewer nodes.

The suffix tree of abaaba\$

T = abaaba\$

- Every leaf is one suffix; the boxed number is the **offset** where that suffix starts in T.
- Every internal node is a place where the text genuinely branches – a substring that is followed by two different characters somewhere in T.
- Reading the edge labels from the root to a leaf spells that suffix exactly.



	count
leaves	7 = m
internal nodes (incl. root)	4
total	11
suffix trie, for comparison	22

How many nodes can a suffix tree have?

Write L for the number of leaves, I for the number of internal nodes, E for edges.

$$E = L + I - 1$$

a tree on $L + I$ nodes has one fewer edge

$$E \geq 2I$$

compactness: every internal node has ≥ 2 children

$$L + I - 1 \geq 2I \Rightarrow I \leq L - 1$$

combine the two

$$L \leq m$$

at most one leaf per suffix, and T has m suffixes

$$I \leq m - 1$$

substitute

$$E = L + I - 1 \leq 2m - 2$$

substitute again

$$E + L + I \leq 4m - 3 \in O(m)$$

the whole tree is linear in m

Checked on abaaba\$: $L = 7, I = 4, E = 10 \leq 4m - 3 = 25$.

...but the tree is not $O(m)$ yet

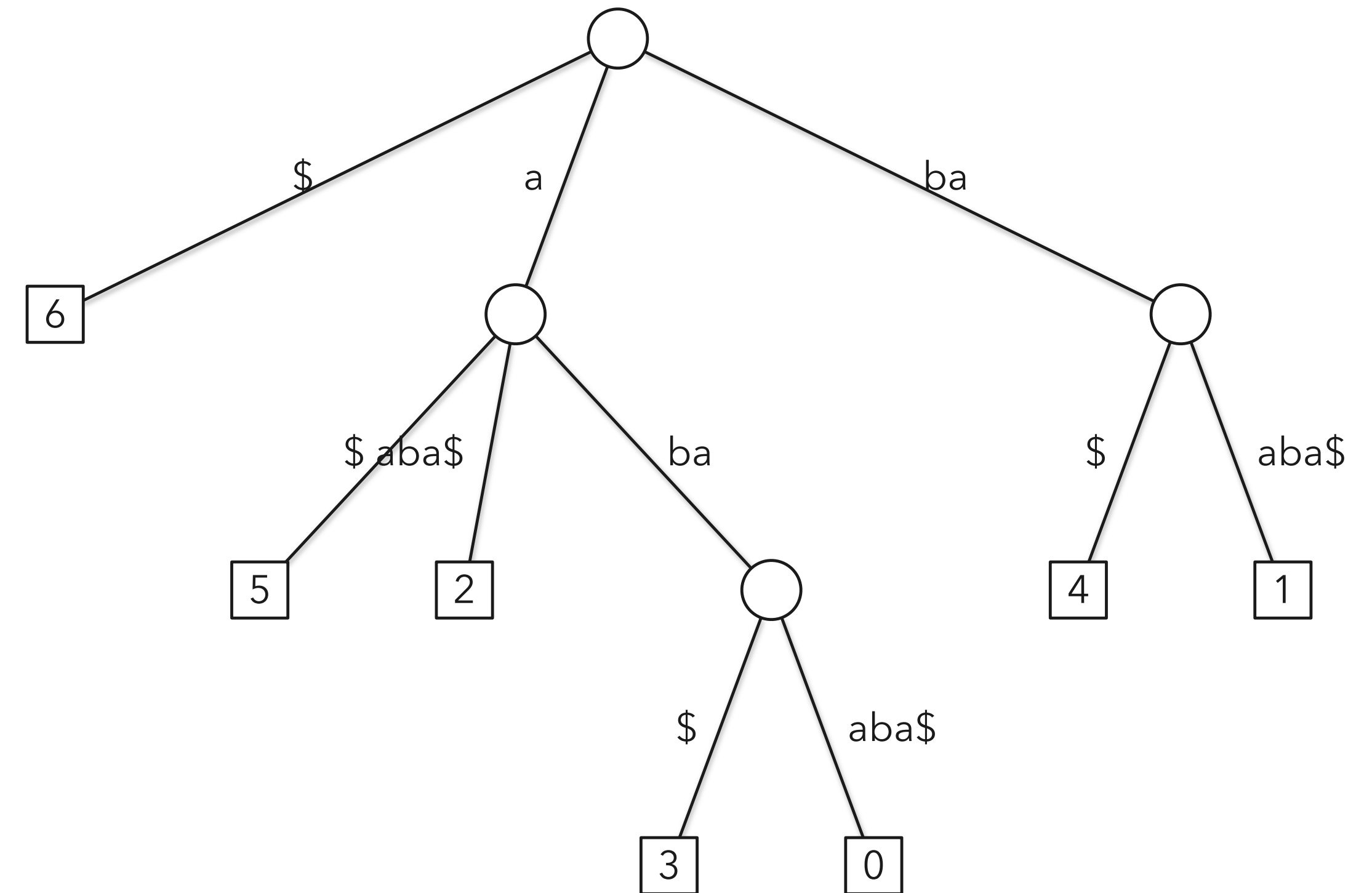
$O(m)$ nodes and $O(m)$ edges – but how much space do the edge labels take?

edge labels of the suffix tree of **abaaba\$**

aba\$ aba\$ aba\$ ba ba \$ a
\$ \$ \$

21 characters of edge label for a text of 7 characters.

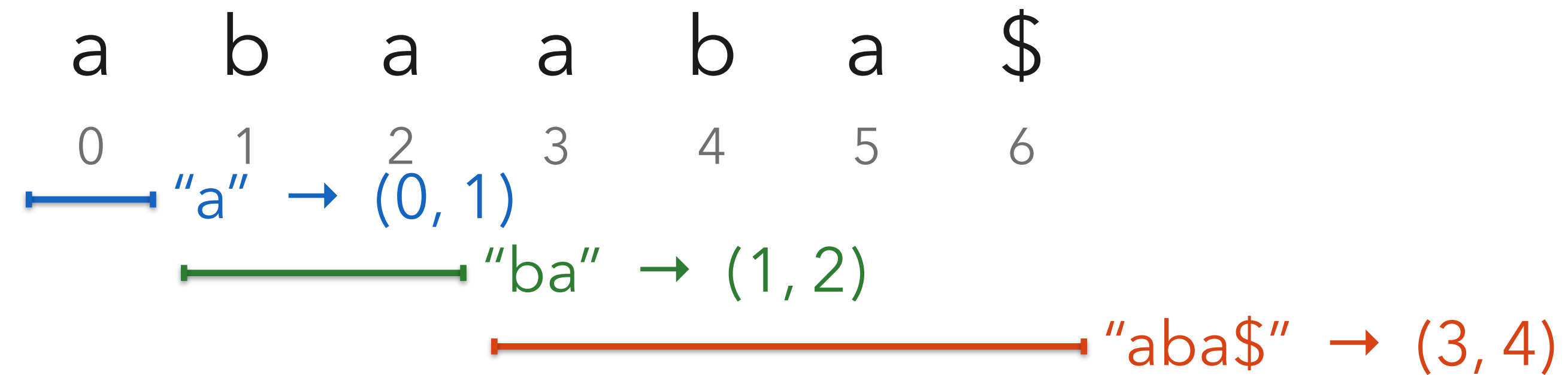
- In the worst case the labels total $\Theta(m^2)$ characters – think of $a^n b^n$ again.
- **Storing labels as strings throws away everything coalescing won.**



So: is the total size $O(m)$ now? No.

Idea 2: labels as windows into T

Keep T itself, and replace every edge label with an (offset, length) pair – a pointer into the text we already have.



Two integers per edge, $O(m)$ edges $\rightarrow O(m)$
space, for good.

- An edge no longer stores characters; it stores *where to look* for them.
- Reading an edge label costs one lookup into T, so queries are unaffected.

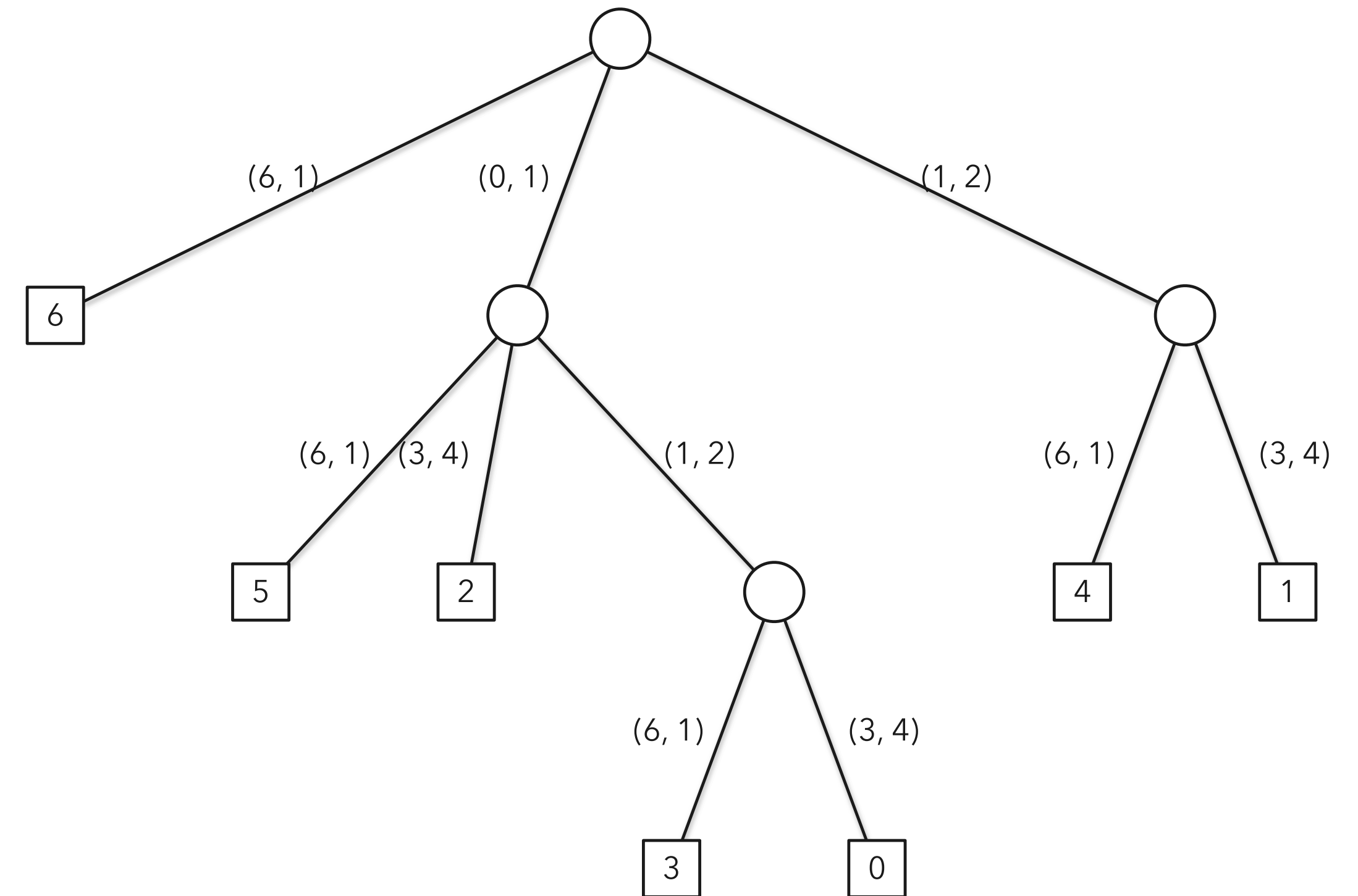
Suffix tree: offsets everywhere

$T = \text{abaaba\$}$

- Each edge carries an (offset, length) pair instead of a string.
- Each leaf carries the offset where its suffix begins – so once we reach a node, the answers to “where does S occur?” are sitting in the leaves below it.
- Node labels – the strings themselves – are never stored; they are recovered by walking down from the root.

The tree is $O(m)$, the labels are $O(m)$, T is m characters
– the whole index is $O(m)$.

Small print: each integer needs $\lceil \log_2 m \rceil$ bits, so “ $O(m)$ ” means $O(m)$ machine words. For any genome, 64-bit words are plenty and we ignore the factor.



Building a suffix tree

method	how	time	space
naive 1	build the suffix trie, then coalesce and relabel	$O(m^2)$	$O(m^2)$
naive 2	start from the longest suffix, add the rest one at a time	$O(m^2)$	$O(m)$
Ukkonen	amortised extension, using suffix links	$O(m)$	$O(m)$

- Naive method 2 is written up in Gusfield §5.4 – worth reading, because it makes clear what Ukkonen has to avoid.
- Ukkonen also has the **online** property: if T arrives one character at a time, the tree is updated after each arrival, still in $O(m)$ total.
- We will not prove it here – see Gusfield Ch. 6.

Ukkonen, E. "On-line construction of suffix trees." Algorithmica 14.3 (1995): 249-260.

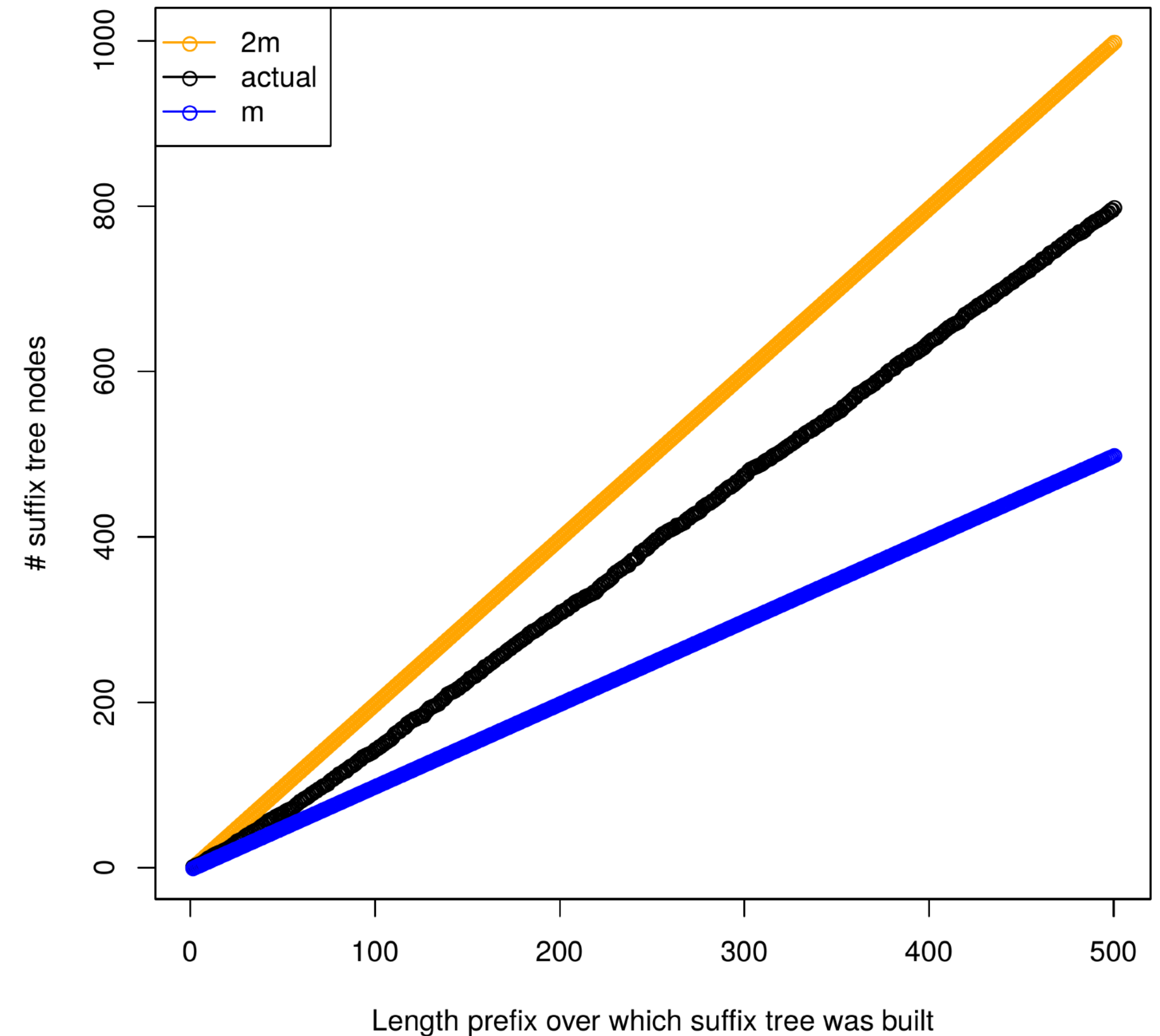
Suffix tree: in practice

- Suffix trees over the same 500 prefixes of the λ -phage genome.
- Black is the actual node count; orange is $2m$, blue is m . It sits between them, exactly as the counting argument promised.

At $m = 500$ the suffix tree has under 800 nodes.

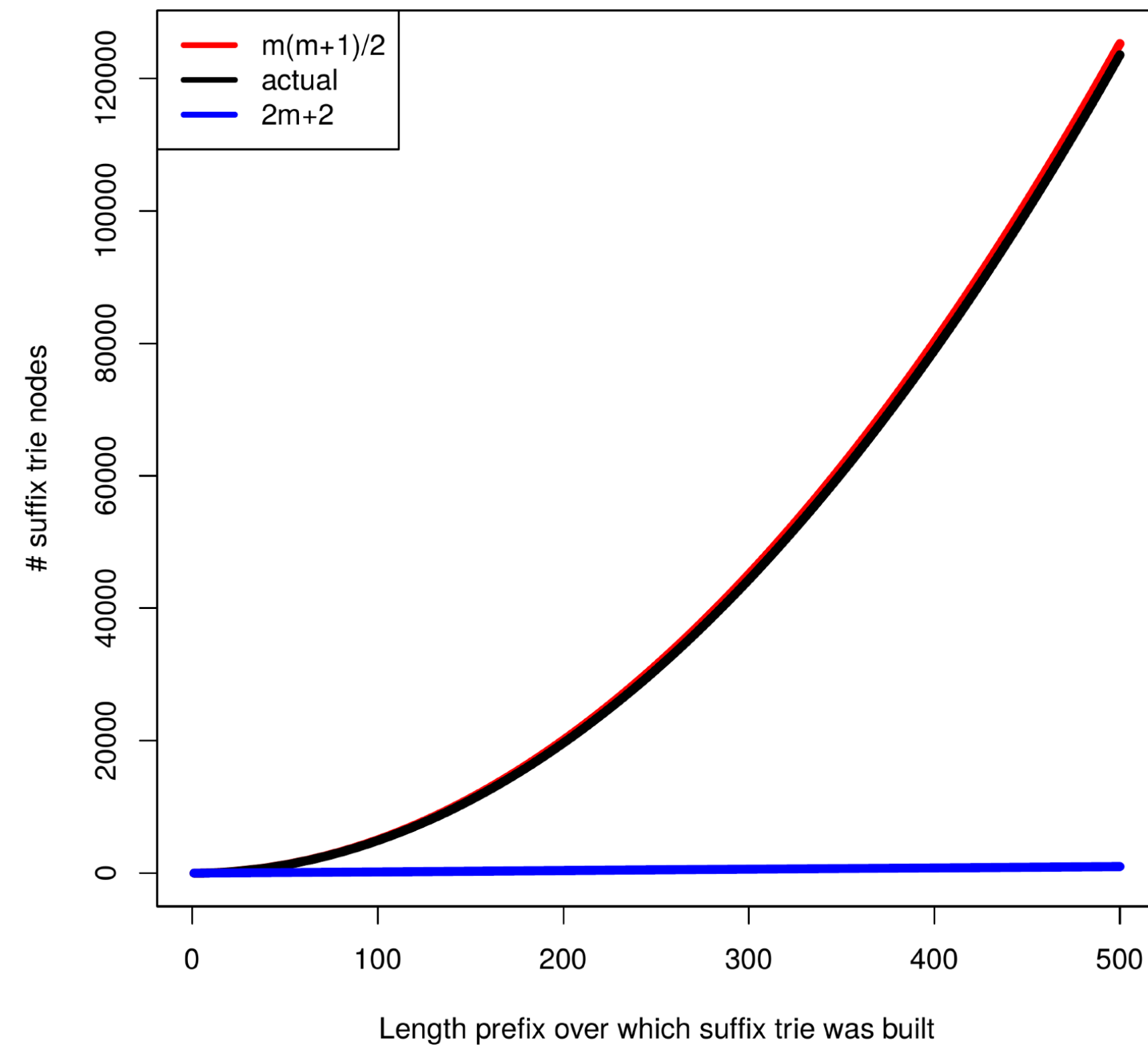
The suffix trie needed 123,000.

Note the vertical axis: it runs to 1,000, not 250,000.



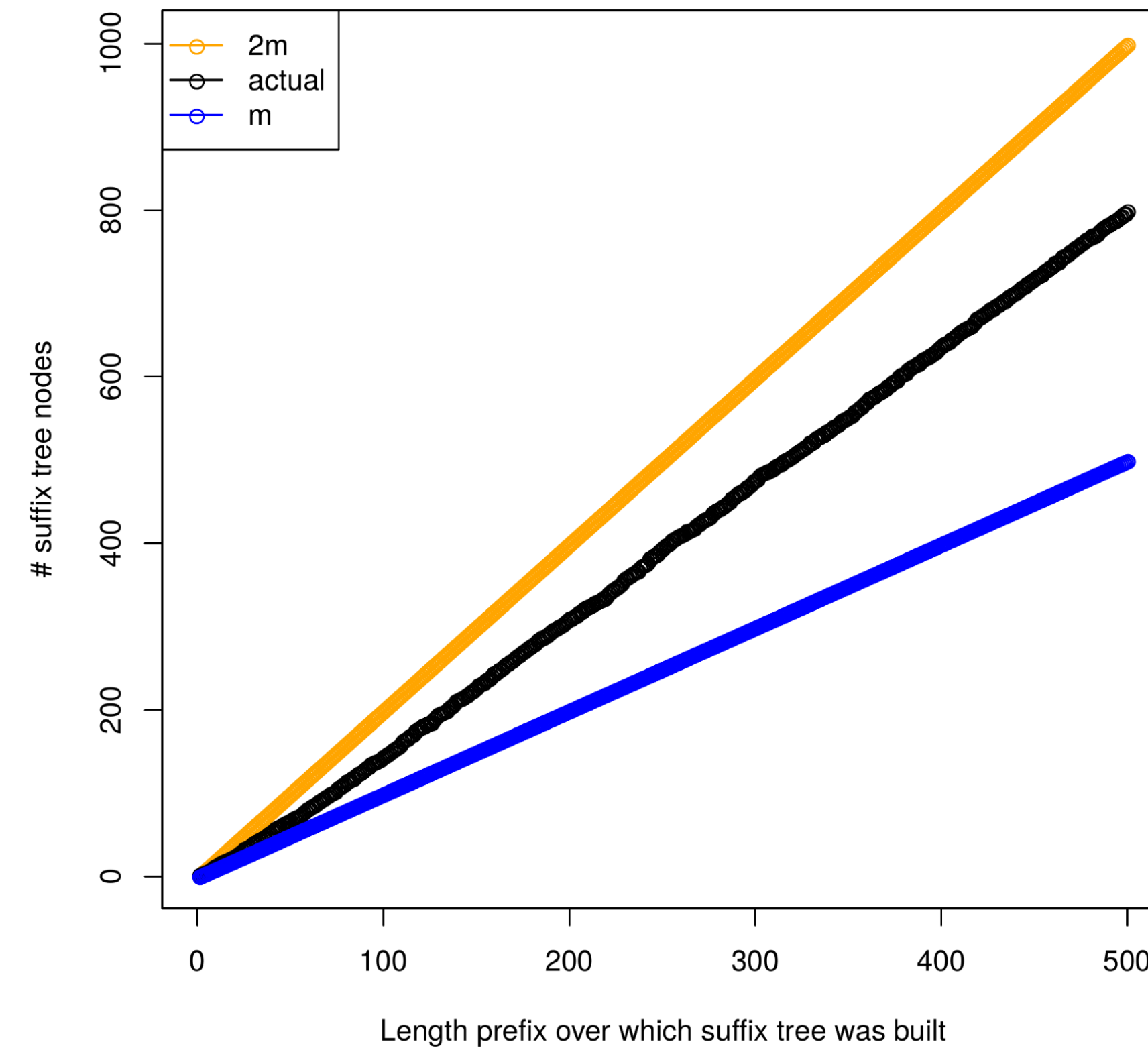
The same text, the same information

Suffix trie



> 100 K nodes

Suffix tree



< 1 K nodes

More than 100× fewer nodes – and $O(m)$ instead of $O(m^2)$.

Applications

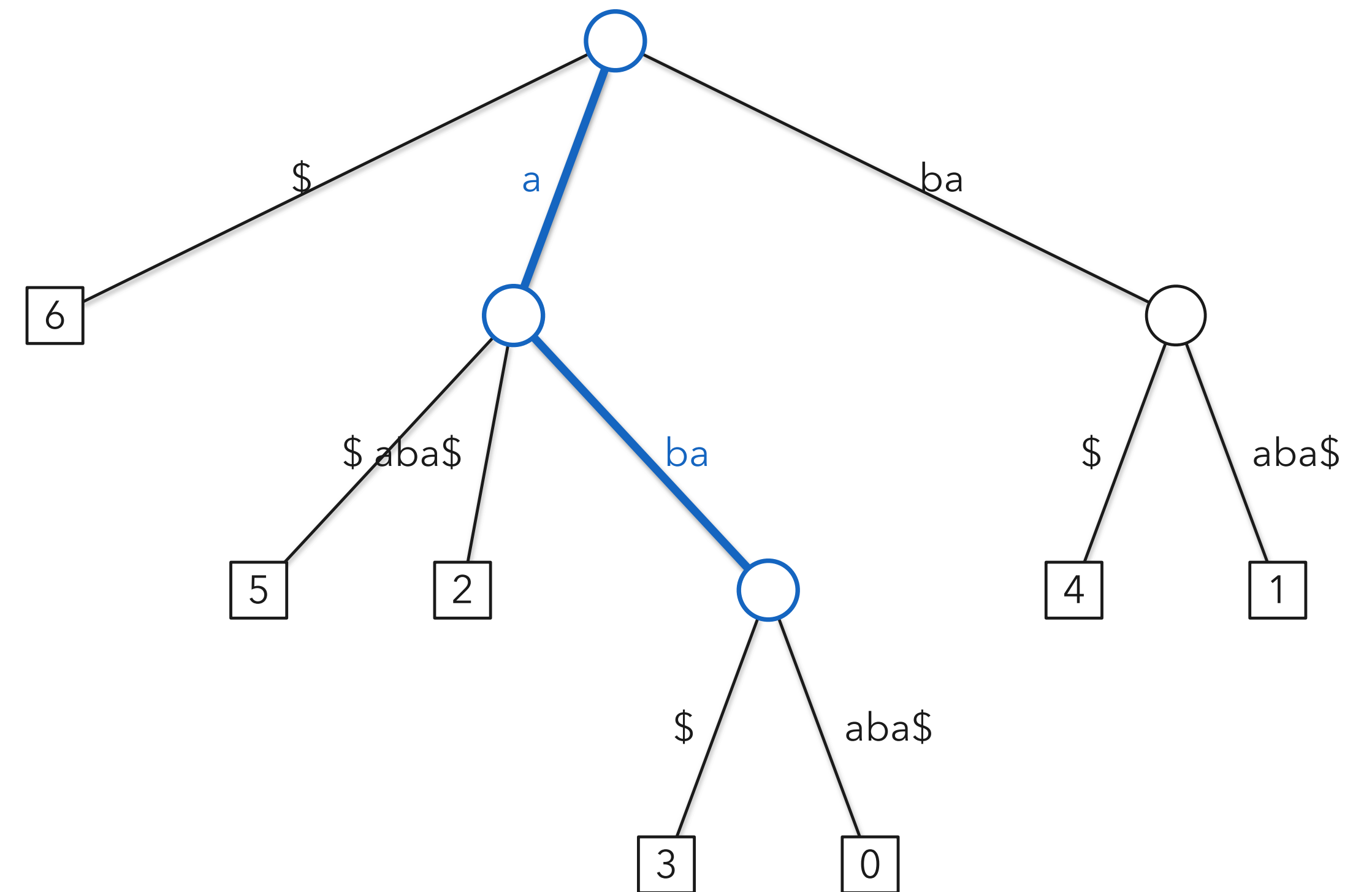
What the tree is good for once you have it

One difference: implicit nodes

- Querying a suffix tree is the same walk as before, with one wrinkle: a query can run out in the **middle of an edge**, because edges now carry whole strings.
- Such a position is an **implicit node**: a perfectly good node of the suffix trie that the tree doesn't store.
- So: match along edges; if you exhaust S you have your answer, real node or not.

$S = ab$

ends one character into the edge labelled "ba",
at an implicit node –
still a substring of T.



the blue path stops inside an edge – there is no node there

Suffix tree: the same four questions

query	suffix trie	suffix tree
is S a substring?	walk; don't fall off	same, may end mid-edge
is S a suffix?	node has a \$ child	locus has an outgoing \$
how often does S occur?	leaves below node(S)	leaves below the locus of S
where does S occur?	leaf offsets below node(S)	leaf offsets below the locus of S
longest repeated substring	deepest node with ≥ 2 children	internal node of greatest string-depth

The "locus" of S is the node at or just below where the walk for S ends. Everything else carries over unchanged – which is the point: we bought a factor of m in space and gave up nothing.

Reporting every match

- Step 1: walk down the path spelling S. If you fall off, there are no matches.
- Step 2: visit every leaf below where you stopped, and report its offset.

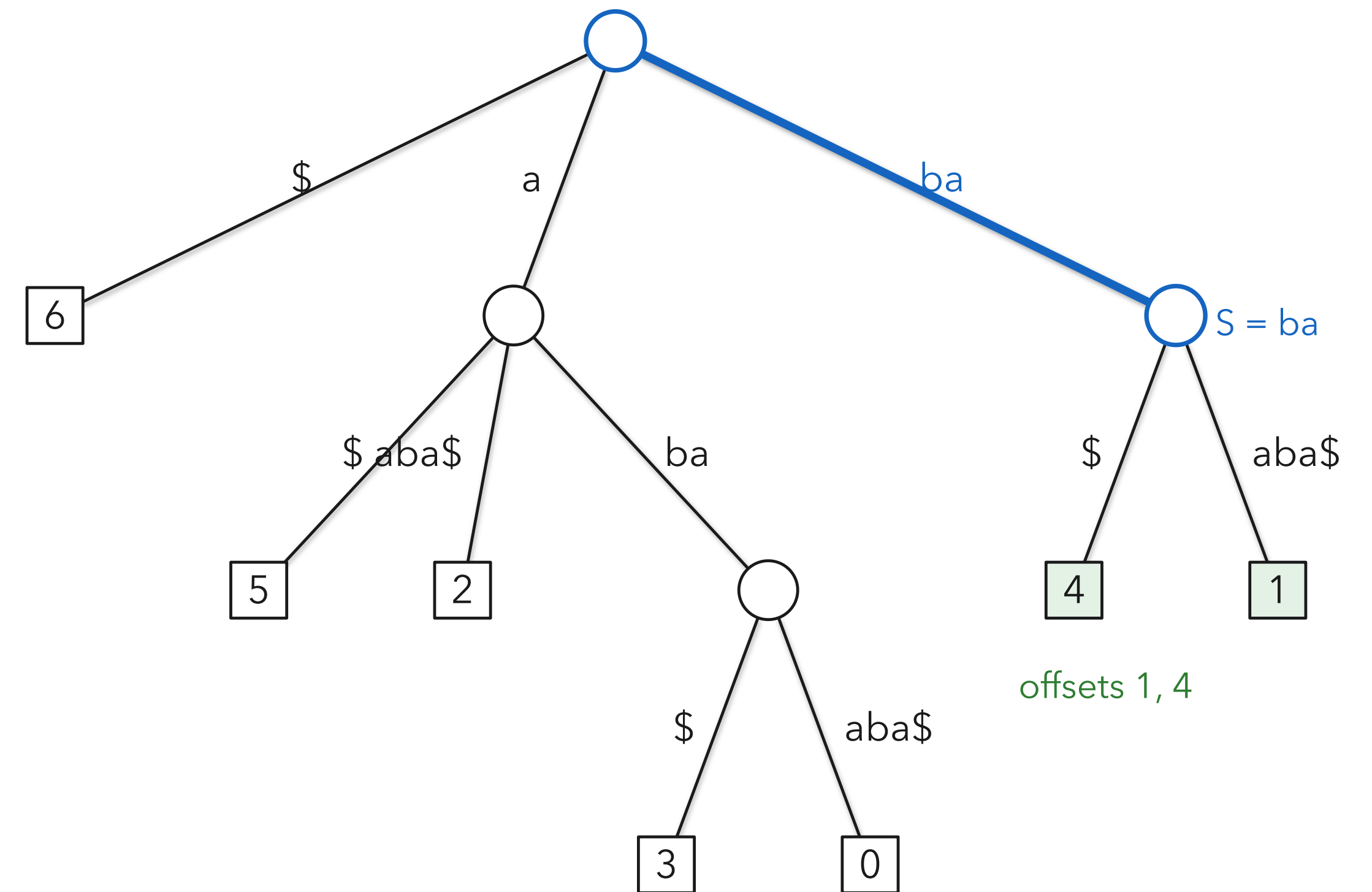
Step 1 costs $O(n)$ – one step per character of S.

Step 2 costs $O(k)$ – one step per match reported.

$$O(n + k)$$

linear in the query, plus linear in the
answer –
independent of m.

A naive scan would cost $O(mn)$ and would touch the whole text every time. Here T is never read, except to check an edge label.



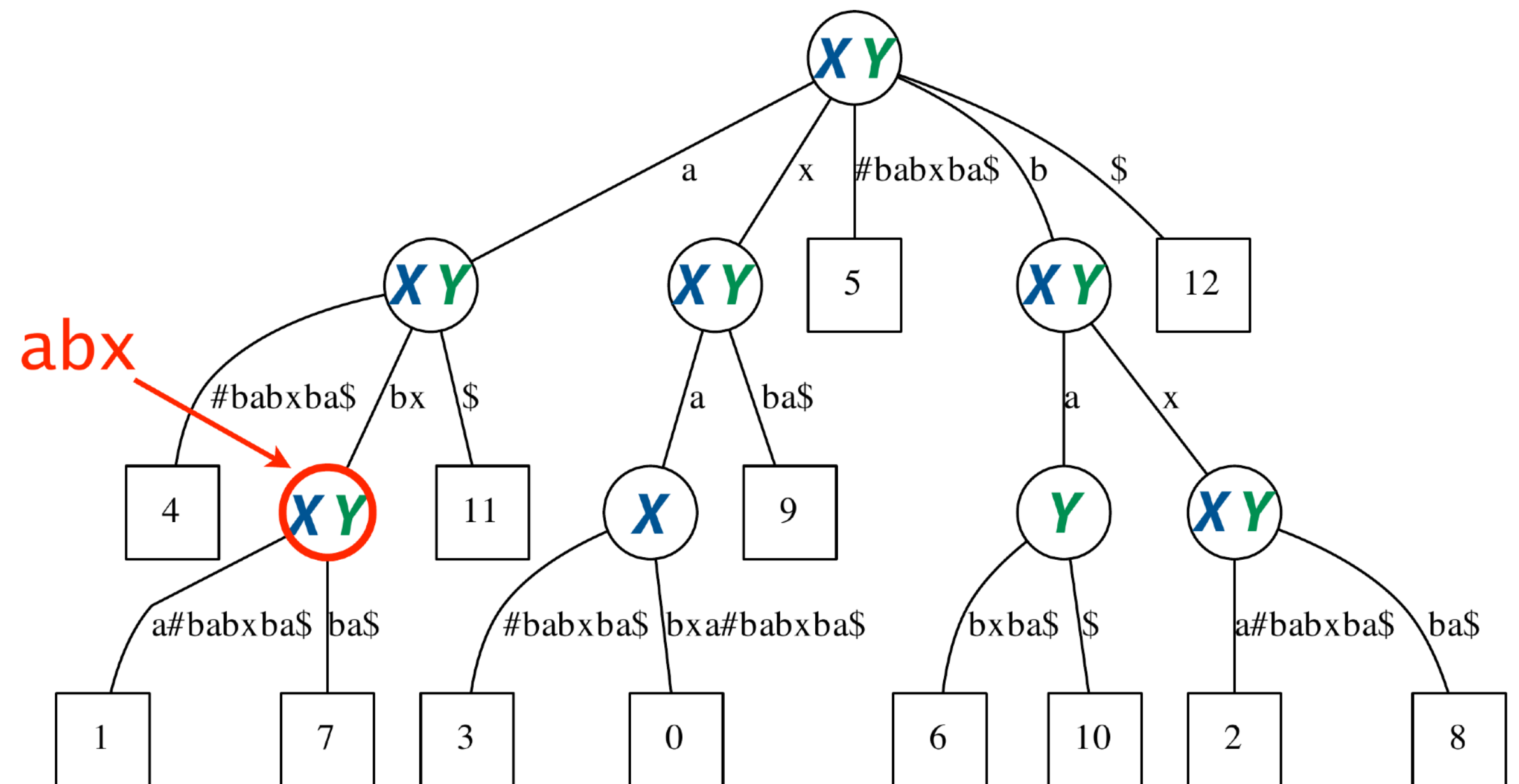
Application: longest common substring

- Build a suffix tree over **X # Y \$**, where # and \$ are two different terminators.
- Annotate each node with whether the leaves below it hold suffixes of X, of Y, or of both.
- The deepest node annotated **both** has the LCS as its label.
- **$O(|X| + |Y|)$ time and space.**

X = xabxa

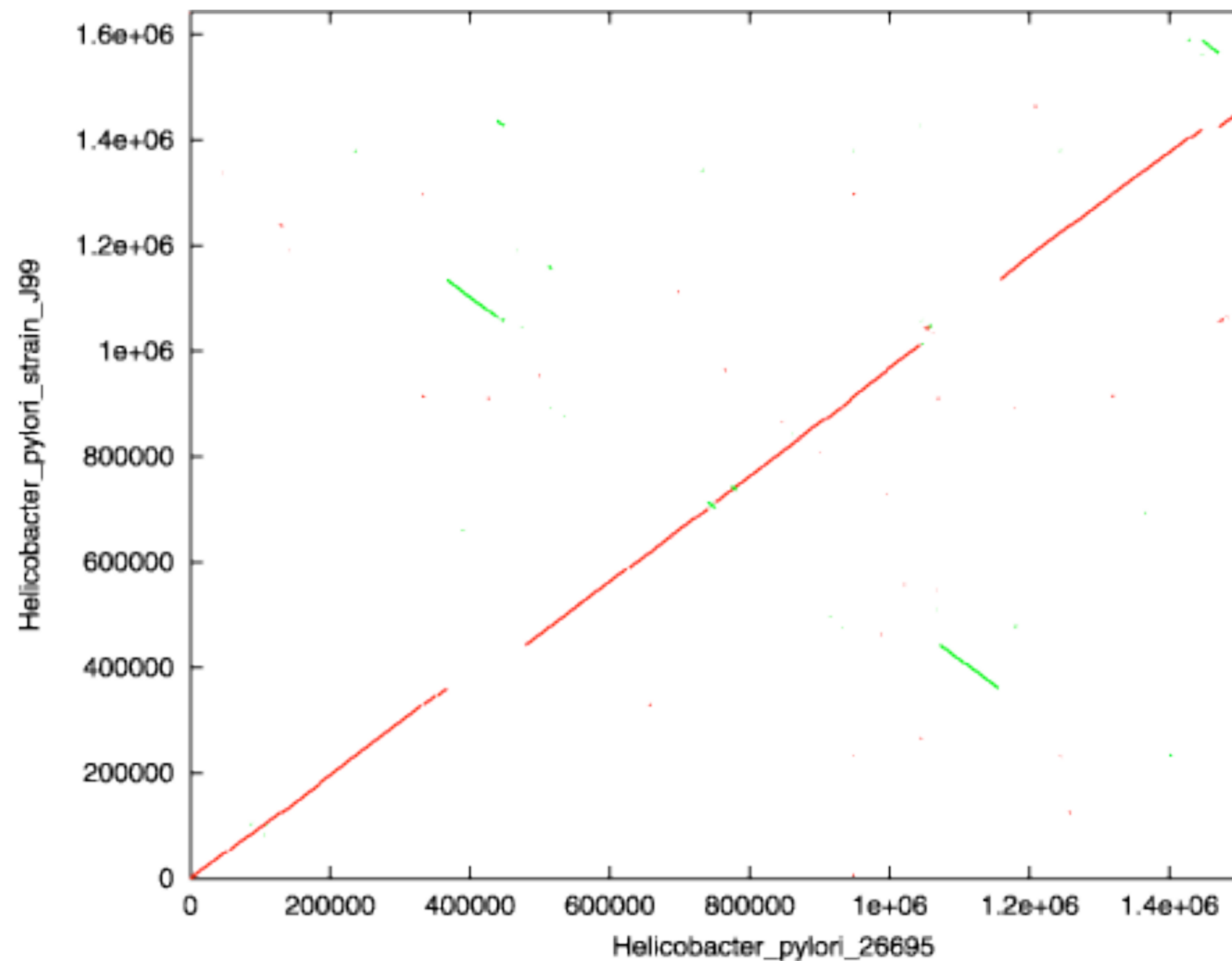
Y = babxba

answer: **abx**



suffix tree of xabxa#babxba\$

Application: comparing whole genomes



- A suffix tree over several strings at once is a **generalized suffix tree** (Gusfield §6.4). The $X\#Y\$$ trick is the two-string case.
- Each dot here is a **maximal unique match** – a long substring occurring exactly once in each of two *H. pylori* strains.
- Red: the match is between like strands.
- Green: opposite strands – an inversion.
- The near-continuous red diagonal is the shared backbone; the breaks and green streaks are the rearrangements. All of it falls out of one index.

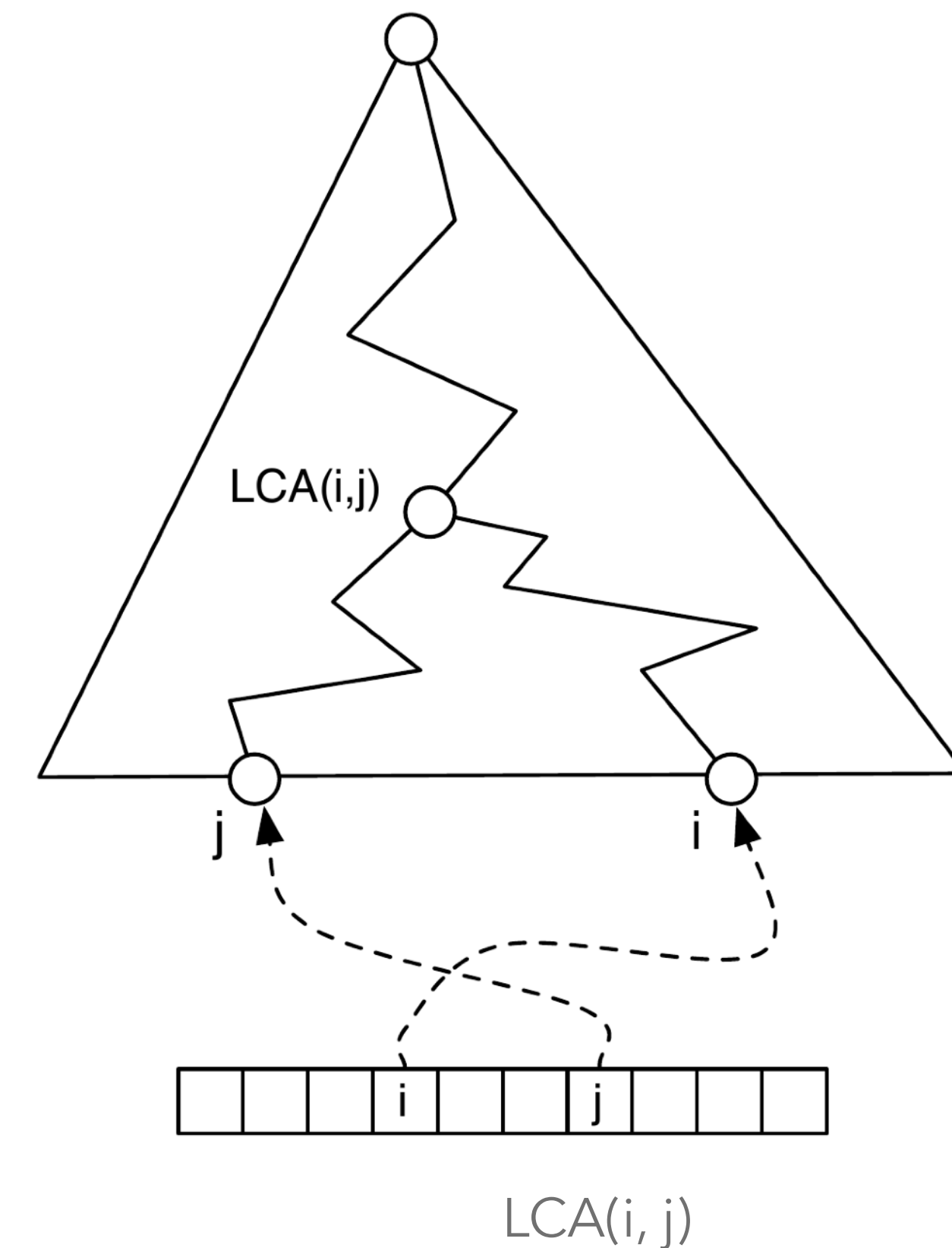
Application: longest common extension

Given S and T , answer many queries (i, j) : how far do S -from- i and T -from- j agree?

- Build a generalized suffix tree for S and T . $O(|S| + |T|)$
- Preprocess it so lowest common ancestors are found in $O(1)$, via range-minimum queries. $O(|S| + |T|)$
- Store an array mapping each suffix number to its leaf. $O(|S| + |T|)$
- **Then $LCE(i, j) = \text{string-depth of LCA}(\text{leaf } i, \text{leaf } j)$. $O(1)$ per query**

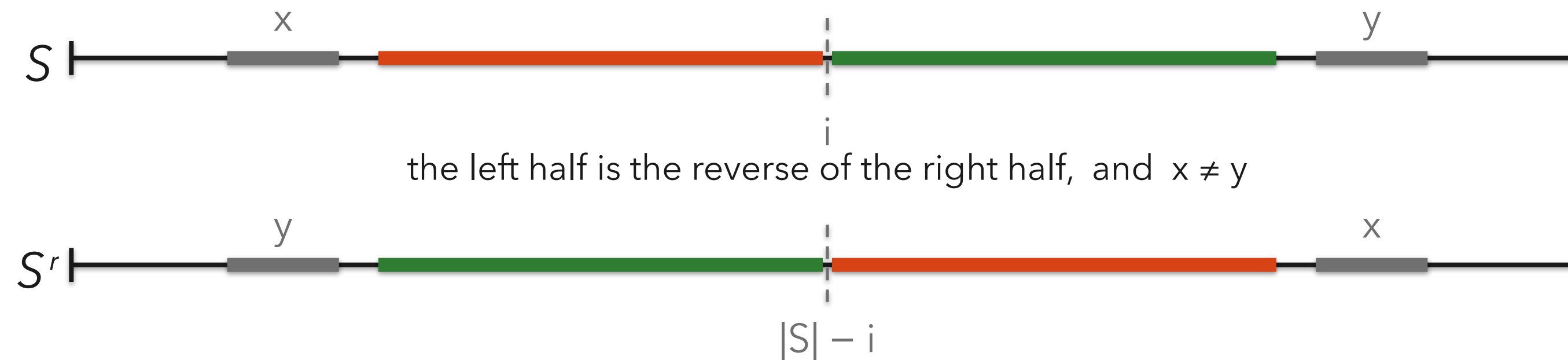
All the work happens once, up front. Each of the many queries is constant time.

Bender & Farach-Colton, "The LCA problem revisited" (2000).



Application: all maximal palindromes

A maximal even palindrome at i : the longest block whose left half is the reverse of its right half.



Reversing S swaps the two halves, so the palindrome centred at i in S is the one centred at $|S| - i$ in S^r – and its radius is how far the two agree.

- Build S^r , the reverse of S . $O(|S|)$
- Preprocess S and S^r for $O(1)$ LCE queries. $O(|S|)$
- For each i , $\text{LCE}(i, |S| - i)$ is the radius. $O(1)$ each
- Odd lengths: centre between two characters.

Total: $O(|S|)$ for every maximal palindrome in S .
worked instance: $S = \text{cabbac}$, $i = 3$, radius 3
→ cabbac

In practice

What it costs when T is a genome

MUMmer: index once, then match

```
(base) rob@fermi mummer-4.0.0rc1 % ./mummer -maxmatch data/chr1.fa data/line1.fa | head -n100
> NC_000001.11:62194849-62212328
62194849      1      17480
62195013     125     21
62194973     165     21
62195059     171     35
62195189     181     23
62195019     211     35
62195189     221     23
62195029     341     23
62195069     341     23
227144292    621     22
226080010    623     45
150924429    623     29
109966311    624     44
160754930    624     44
13373677     624     44
28521220     624     44
195870013    624     44
155102021    624     31
39601269     624     31
35187449     624     30
168153630    624     30
155228268    624     28
117635680    624     24
37579266     624     21
155399348    624     20
176178570    624     20
174043843    624     20
23923078     624     20
169606035    624     20
28188030     624     20
232504257    624     20
27791535     625     37
112543271    627     34
150082229    628     40
154581494    628     40
228796306    628     40
43941439     628     36
45489512     628     34
145504125    628     34
148077279    628     34
145822049    628     34
155230233    628     32
```

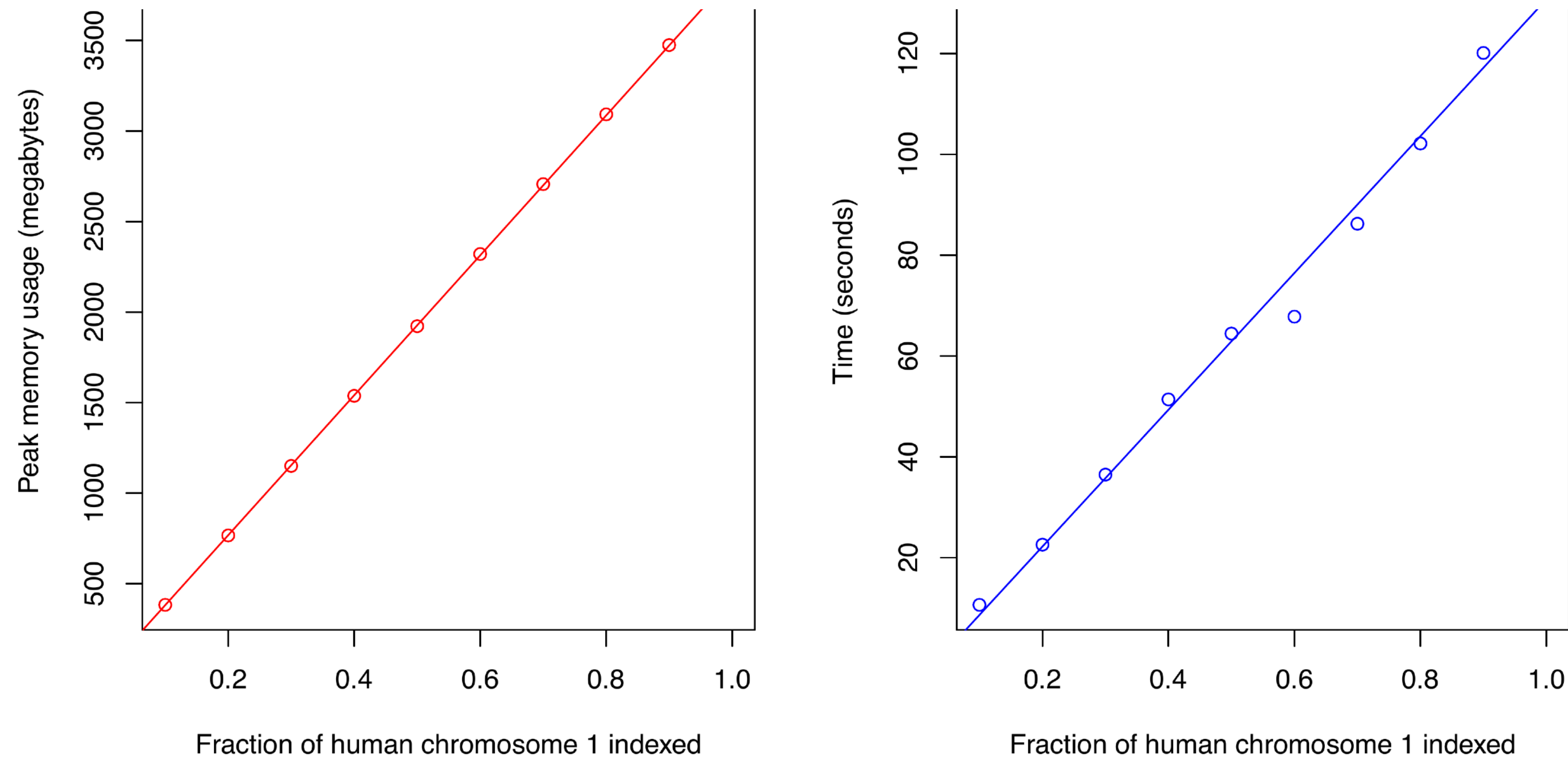
- Text: human chromosome 1. Pattern: a LINE-1 element.
- MUMmer first builds the index over T – about a minute on my machine.
- It then reports maximal matches between T and P. That phase is very fast, and there are many matches.

column	meaning
1	offset in T
2	offset in P
3	length of the exact match

The asymmetry is the whole argument for indexing: one slow build, then arbitrarily many cheap queries.

MUMmer: how it scales

Time and peak memory while indexing increasing fractions of human chromosome 1.



Both are straight lines: $O(m)$ time and $O(m)$ space, as promised. Whole chromosome 1: 2 m 14 s, and 3.94 GB of memory.

...and then you try the whole genome

```
mummer: suffix tree construction failed: textlen=3101804822  
larger than maximal textlen=536870908
```

- **$O(m)$ is the right asymptotic answer, and it is still not enough.**
- At 15.76 bytes per base, the human genome (3.1 Gb) would need roughly 49 GB of memory.
- The constant in front of the m is what decides whether a data structure is usable, and for suffix trees it is large.

3.94 GB / 250 Mnt $\approx$ 15.76 bytes per base $\rightarrow$ 3.1 Gb $\times$ 15.76 $\approx$ 49 GB

The constant factor

- Why so big? Each node needs child pointers, a parent or suffix link, and two integers for its edge label – all of them $\lceil \log_2 m \rceil$ -bit quantities.
- How big exactly depends entirely on the implementation:

implementation	bytes per input base
MUMmer v3.32, measured above	≈ 15.76
best used in practice	≈ 12.5
best reported in the literature	≈ 8.5

Kurtz, S. "Reducing the space requirement of suffix trees." Software: Practice and Experience 29.13 (1999): 1149-1171.

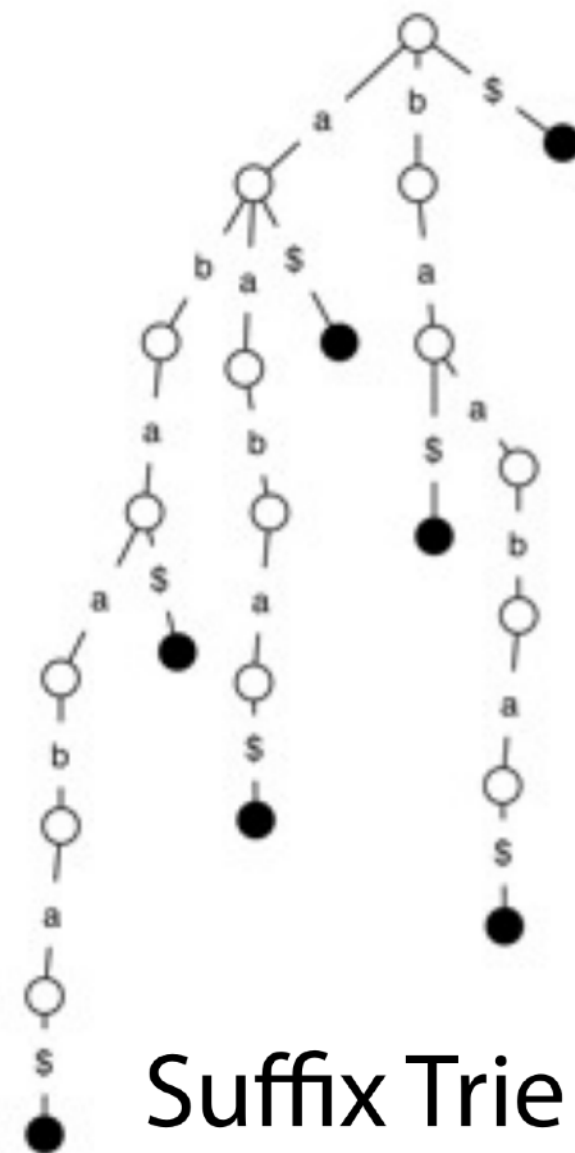
Even at the best reported 8.5 bytes per base, a human genome is ~26 GB. Linear is necessary here, but it is not sufficient.

Summary

	suffix trie	suffix tree
nodes	$O(m^2)$	$O(m)$
edge labels	one character	(offset, length) into T
total space	$O(m^2)$	$O(m)$
build time	$O(m^2)$	$O(m)$ (Ukkonen)
is S a substring?	$O(n)$	$O(n)$
report all k matches	$O(n + k)$	$O(n + k)$
λ -phage, $m = 500$	123,000 nodes	< 800 nodes

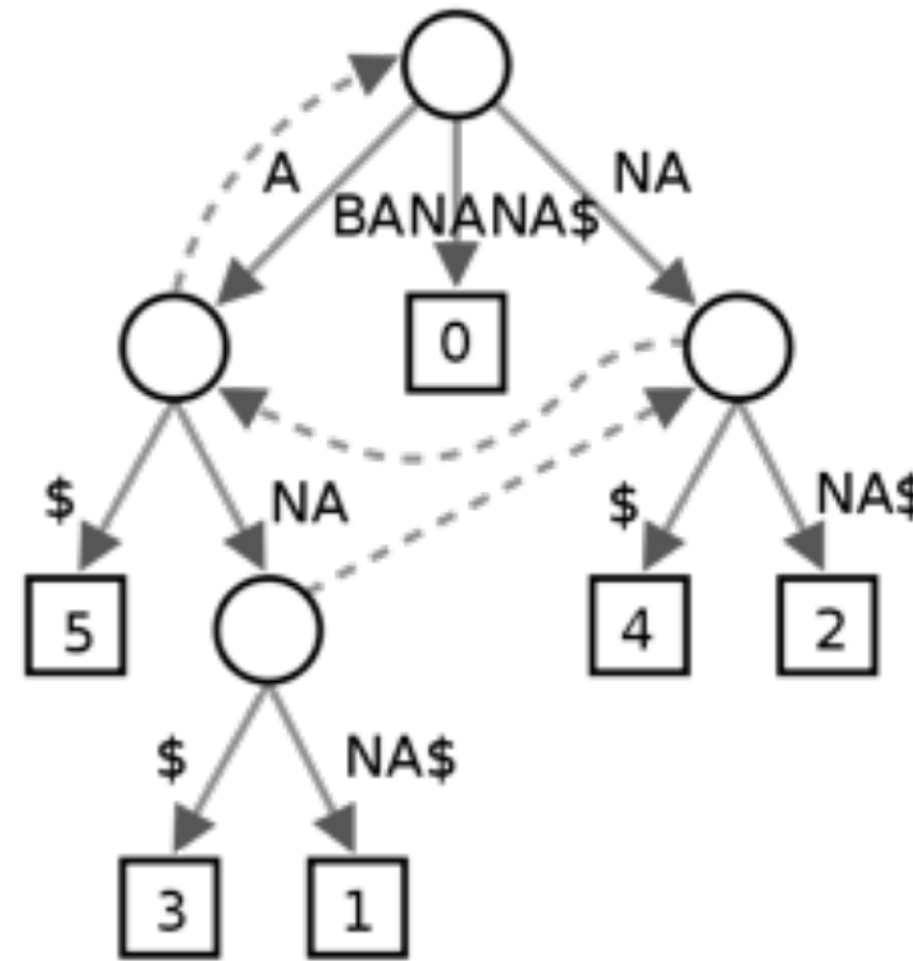
- One structure answers substring, suffix, count and all-occurrences queries in time proportional to the query.
- The naive version is quadratic and unusable; coalescing and offsets make it linear.
- Its memory footprint per base still keeps it off whole genomes.

Where this goes next



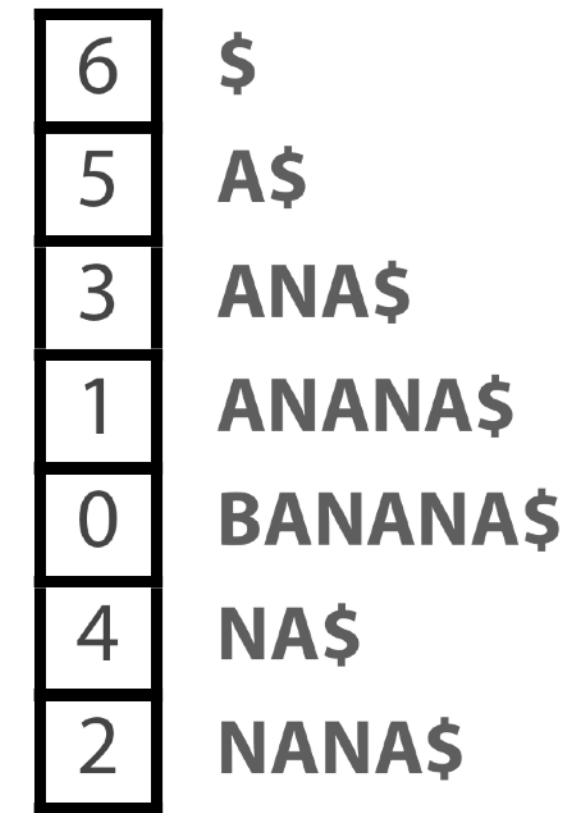
Suffix Trie

✓ **built today**



Suffix Tree

✓ **built today**



Suffix Array

next lecture

\$ BANANA
A \$ BANAN
ANA \$ BAN
ANANA \$ B
BANANA \$
NA \$ BAN A
NANA \$ BA

FM Index

next lecture

- Same information, progressively less space: the suffix array keeps the sorted order and throws the tree away; the FM index compresses T and searches it in place.
- Both keep $O(n)$ per query, at a constant factor small enough for a human genome – which is why every modern read aligner uses one.